

RISC-V External Debug Support

Version 0.13.2

d5029366d59e8563c08b6b9435f82573b603e48e

Editors:

Tim Newsome <tim@sifive.com>, SiFive, Inc.
Megan Wachs <megan@sifive.com>, SiFive, Inc.

Fri Mar 22 09:06:04 2019 -0700

Contributors to all versions of the spec in alphabetical order (please contact editors to suggest corrections): Bruce Ableidinger, Krste Asanović, Allen Baum, Mark Beal, Alex Bradbury, Chuanhua Chang, Zhong-Ho Chen, Monte Dalrymple, Vyacheslav Dyachenko, Peter Egold, Markus Goehrle, Robert Golla, John Hauser, Richard Herveille, Yung-ching Hsiao, Po-wei Huang, Scott Johnson, Jean-Luc Nagel, Aram Nahidipour, Rishiyur Nikhil, Gajinder Panesar, Deepak Panwar, Antony Pavlov, Klaus Kruse Pedersen, Ken Pettit, Joe Rahmeh, Gavin Stark, Wesley Terpstra, Jan-Willem van de Waerdt, Stefan Wallentowitz, Ray Van De Walker, Andrew Waterman, Andy Wright, and Bryan Wyatt.

Contents

1	Introduction	1
1.1	Terminology	1
1.1.1	Context	1
1.1.2	Versions	2
1.2	About This Document	2
1.2.1	Structure	2
1.2.2	Register Definition Format	2
1.2.2.1	Long Name (<code>shortname</code> , at 0x123)	2
1.3	Background	3
1.4	Supported Features	3
2	System Overview	5
3	Debug Module (DM)	7
3.1	Debug Module Interface (DMI)	8
3.2	Reset Control	8
3.3	Selecting Harts	9
3.3.1	Selecting a Single Hart	9
3.3.2	Selecting Multiple Harts	9
3.4	Hart States	9
3.5	Run Control	10
3.6	Abstract Commands	11

3.6.1	Abstract Command Listing	12
3.6.1.1	Access Register	12
3.6.1.2	Quick Access	14
3.6.1.3	Access Memory	14
3.7	Program Buffer	16
3.8	Overview of States	16
3.9	System Bus Access	18
3.10	Minimally Intrusive Debugging	18
3.11	Security	19
3.12	Debug Module Registers	19
3.12.1	Debug Module Status (<code>dmstatus</code> , at 0x11)	20
3.12.2	Debug Module Control (<code>dmcontrol</code> , at 0x10)	22
3.12.3	Hart Info (<code>hartinfo</code> , at 0x12)	25
3.12.4	Hart Array Window Select (<code>hawindowse1</code> , at 0x14)	26
3.12.5	Hart Array Window (<code>hawindow</code> , at 0x15)	26
3.12.6	Abstract Control and Status (<code>abstractcs</code> , at 0x16)	27
3.12.7	Abstract Command (<code>command</code> , at 0x17)	28
3.12.8	Abstract Command Autoexec (<code>abstractauto</code> , at 0x18)	29
3.12.9	Configuration String Pointer 0 (<code>confstrptr0</code> , at 0x19)	29
3.12.10	Next Debug Module (<code>nextdm</code> , at 0x1d)	30
3.12.11	Abstract Data 0 (<code>data0</code> , at 0x04)	30
3.12.12	Program Buffer 0 (<code>progbuf0</code> , at 0x20)	30
3.12.13	Authentication Data (<code>authdata</code> , at 0x30)	31
3.12.14	Halt Summary 0 (<code>haltsum0</code> , at 0x40)	31
3.12.15	Halt Summary 1 (<code>haltsum1</code> , at 0x13)	31
3.12.16	Halt Summary 2 (<code>haltsum2</code> , at 0x34)	32
3.12.17	Halt Summary 3 (<code>haltsum3</code> , at 0x35)	32
3.12.18	System Bus Access Control and Status (<code>sbcsc</code> , at 0x38)	32

3.12.19	System Bus Address 31:0 (sbaddress0 , at 0x39)	34
3.12.20	System Bus Address 63:32 (sbaddress1 , at 0x3a)	35
3.12.21	System Bus Address 95:64 (sbaddress2 , at 0x3b)	35
3.12.22	System Bus Address 127:96 (sbaddress3 , at 0x37)	36
3.12.23	System Bus Data 31:0 (sbdata0 , at 0x3c)	36
3.12.24	System Bus Data 63:32 (sbdata1 , at 0x3d)	37
3.12.25	System Bus Data 95:64 (sbdata2 , at 0x3e)	37
3.12.26	System Bus Data 127:96 (sbdata3 , at 0x3f)	38
4	RISC-V Debug	39
4.1	Debug Mode	39
4.2	Load-Reserved/Store-Conditional Instructions	40
4.3	Wait for Interrupt Instruction	40
4.4	Single Step	40
4.5	Reset	41
4.6	dret Instruction	41
4.7	XLEN	41
4.8	Core Debug Registers	41
4.8.1	Debug Control and Status (dcsr , at 0x7b0)	42
4.8.2	Debug PC (dpc , at 0x7b1)	44
4.8.3	Debug Scratch Register 0 (dscratch0 , at 0x7b2)	45
4.8.4	Debug Scratch Register 1 (dscratch1 , at 0x7b3)	45
4.9	Virtual Debug Registers	45
4.9.1	Privilege Level (priv , at virtual)	45
5	Trigger Module	47
5.1	Native M-Mode Triggers	48
5.2	Trigger Registers	48
5.2.1	Trigger Select (tselect , at 0x7a0)	49

5.2.2	Trigger Data 1 (<code>tdata1</code> , at 0x7a1)	50
5.2.3	Trigger Data 2 (<code>tdata2</code> , at 0x7a2)	50
5.2.4	Trigger Data 3 (<code>tdata3</code> , at 0x7a3)	51
5.2.5	Trigger Info (<code>tinfo</code> , at 0x7a4)	51
5.2.6	Trigger Control (<code>tcontrol</code> , at 0x7a5)	51
5.2.7	Machine Context (<code>mcontext</code> , at 0x7a8)	52
5.2.8	Supervisor Context (<code>scontext</code> , at 0x7aa)	52
5.2.9	Match Control (<code>mcontrol</code> , at 0x7a1)	53
5.2.10	Instruction Count (<code>icount</code> , at 0x7a1)	58
5.2.11	Interrupt Trigger (<code>itrigger</code> , at 0x7a1)	59
5.2.12	Exception Trigger (<code>etrigger</code> , at 0x7a1)	60
5.2.13	Trigger Extra (RV32) (<code>textra32</code> , at 0x7a3)	60
5.2.14	Trigger Extra (RV64) (<code>textra64</code> , at 0x7a3)	61
6	Debug Transport Module (DTM)	62
6.1	JTAG Debug Transport Module	62
6.1.1	JTAG Background	62
6.1.2	JTAG DTM Registers	63
6.1.3	IDCODE (at 0x01)	63
6.1.4	DTM Control and Status (<code>dtmcs</code> , at 0x10)	64
6.1.5	Debug Module Interface Access (<code>dmi</code> , at 0x11)	65
6.1.6	BYPASS (at 0x1f)	66
6.1.7	Recommended JTAG Connector	67
A	Hardware Implementations	69
A.1	Abstract Command Based	69
A.2	Execution Based	69
B	Debugger Implementation	71

B.1	Debug Module Interface Access	71
B.2	Checking for Halted Harts	72
B.3	Halting	72
B.4	Running	72
B.5	Single Step	72
B.6	Accessing Registers	72
B.6.1	Using Abstract Command	72
B.6.2	Using Program Buffer	73
B.7	Reading Memory	73
B.7.1	Using System Bus Access	73
B.7.2	Using Program Buffer	74
B.7.3	Using Abstract Memory Access	75
B.8	Writing Memory	76
B.8.1	Using System Bus Access	76
B.8.2	Using Program Buffer	76
B.8.3	Using Abstract Memory Access	77
B.9	Triggers	78
B.10	Handling Exceptions	79
B.11	Quick Access	79
C	Bug Fixes	80
C.1	0.13.1	80
C.1.1	Resume ack bit is set after resuming	80
C.1.2	aamsize does not affect Argument Width	80
C.1.3	sbddata0 Reads Order of Operations	80
C.1.4	Hart reset behavior when haltreq is set	81
C.1.5	mte only applies when action=0	81
C.1.6	sselect applies to svalue	81

C.1.7	Last trigger example	81
C.2	0.13.2	81
Index		82

List of Figures

2.1 RISC-V Debug System Overview	6
3.1 Run/Halt Debug State Machine	17

List of Tables

1.2	Register Access Abbreviations	3
3.1	Use of Data Registers	11
3.2	Meaning of <code>cmdtype</code>	12
3.3	Abstract Register Numbers	13
3.7	System Bus Data Bits	18
3.8	Debug Module Debug Bus Registers	20
4.1	Core Debug Registers	42
4.3	Virtual address in DPC upon Debug Mode Entry	44
4.4	Virtual Core Debug Registers	45
4.5	Privilege Level Encoding	46
5.1	action encoding	49
5.2	Trigger Registers	49
5.8	Suggested Breakpoint Timings	53
6.1	JTAG DTM TAP Registers	63
6.5	MIPI-10 Connector Diagram	67
6.6	MIPI-20 Connector Diagram	67
6.7	JTAG Connector Pinout	68

Chapter 1

Introduction

When a design progresses from simulation to hardware implementation, a user’s control and understanding of the system’s current state drops dramatically. To help bring up and debug low level software and hardware, it is critical to have good debugging support built into the hardware. When a robust OS is running on a core, software can handle many debugging tasks. However, in many scenarios, hardware support is essential.

This document outlines a standard architecture for external debug support on RISC-V platforms. This architecture allows a variety of implementations and tradeoffs, which is complementary to the wide range of RISC-V implementations. At the same time, this specification defines common interfaces to allow debugging tools and components to target a variety of platforms based on the RISC-V ISA.

System designers may choose to add additional hardware debug support, but this specification defines a standard interface for common functionality.

1.1 Terminology

A *platform* is a single integrated circuit consisting of one or more *components*. Some components may be RISC-V cores, while others may have a different function. Typically they will all be connected to a single system bus. A single RISC-V core contains one or more hardware threads, called *harts*.

DXLEN of a hart is its widest supported XLEN, ignoring the current value of MXL in `misr`.

1.1.1 Context

This document is written to work with:

1. The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 2.2 (the ISA Spec)

2. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10 (the Privileged Spec)

1.1.2 Versions

Version 0.13 of this document was ratified by the RISC-V Foundation’s board. Versions 0.13.*x* are bug fix releases to that ratified specification.

Version 0.14 will be forwards and backwards compatible with Version 0.13.

1.2 About This Document

1.2.1 Structure

This document contains two parts. The main part of the document is the specification, which is given in the numbered sections. The second part of the document is a set of appendices. The information in the appendices is intended to clarify and provide examples, but is not part of the actual specification.

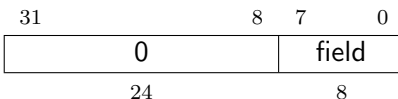
1.2.2 Register Definition Format

All register definitions in this document follow the format shown below. A simple graphic shows which fields are in the register. The upper and lower bit indices are shown to the top left and top right of each field. The total number of bits in the field are shown below it.

After the graphic follows a table which for each field lists its name, description, allowed accesses, and reset value. The allowed accesses are listed in Table 1.2. The reset value is either a constant or “Preset.” The latter means it is an implementation-specific legal value.

Names of registers and their fields are hyperlinks to their definition, and are also listed in the index on page 82.

1.2.2.1 Long Name (shortname, at 0x123)



Field	Description	Access	Reset
field	Description of what this field is used for.	R/W	15

Table 1.2: Register Access Abbreviations

R	Read-only.
R/W	Read/Write.
R/W1C	Read/Write. For each bit in the field, writing 1 clears that bit. Writing 0 has no effect.
W	Write-only. When read this field returns 0.
W1	Write-only. Only writing 1 has an effect.
WARL	Write any, read legal. A debugger may write any value. If a value is unsupported, the implementation converts the value to one that is supported.

1.3 Background

There are several use cases for dedicated debugging hardware, both internal to a CPU core and with an external connection. This specification addresses the use cases listed below. Implementations can choose not to implement every feature, which means some use cases might not be supported.

- Debugging low-level software in the absence of an OS or other software.
- Debugging issues in the OS itself.
- Bootstrapping a system to test, configure, and program components before there is any executable code path in the system.
- Accessing hardware on a system without a working CPU.

In addition, even without a hardware debugging interface, architectural support in a RISC-V CPU can aid software debugging and performance analysis by allowing hardware triggers and breakpoints.

1.4 Supported Features

The debug interface described in this specification supports the following features:

1. All hart registers (including CSRs) can be read/written.
2. Memory can be accessed either from the hart's point of view, through the system bus directly, or both.
3. RV32, RV64, and future RV128 are all supported.
4. Any hart in the platform can be independently debugged.
5. A debugger can discover almost¹ everything it needs to know itself, without user configuration.

¹Notable exceptions include information about the memory map and peripherals.

6. Each hart can be debugged from the very first instruction executed.
7. A RISC-V hart can be halted when a software breakpoint instruction is executed.
8. Hardware single-step can execute one instruction at a time.
9. Debug functionality is independent of the debug transport used.
10. The debugger does not need to know anything about the microarchitecture of the harts it is debugging.
11. Arbitrary subsets of harts can be halted and resumed simultaneously. (Optional)
12. Arbitrary instructions can be executed on a halted hart. That means no new debug functionality is needed when a core has additional or custom instructions or state, as long as there exist programs that can move that state into GPRs. (Optional)
13. Registers can be accessed without halting. (Optional)
14. A running hart can be directed to execute a short sequence of instructions, with little overhead. (Optional)
15. A system bus master allows memory access without involving any hart. (Optional)
16. A RISC-V hart can be halted when a trigger matches the PC, read/write address/data, or an instruction opcode. (Optional)

This document does not suggest a strategy or implementation for hardware test, debugging or error detection techniques. Scan, BIST, etc. are out of scope of this specification, but this specification does not intend to limit their use in RISC-V systems.

It is possible to debug code that uses software threads, but there is no special debug support for it.

Chapter 2

System Overview

Figure 2.1 shows the main components of External Debug Support. Blocks shown in dotted lines are optional.

The user interacts with the Debug Host (e.g. laptop), which is running a debugger (e.g. gdb). The debugger communicates with a Debug Translator (e.g. OpenOCD, which may include a hardware driver) to communicate with Debug Transport Hardware (e.g. Olimex USB-JTAG adapter). The Debug Transport Hardware connects the Debug Host to the Platform's Debug Transport Module (DTM). The DTM provides access to one or more Debug Modules (DMs) using the Debug Module Interface (DMI).

Each hart in the platform is controlled by exactly one DM. Harts may be heterogeneous. There is no further limit on the hart-DM mapping, but usually all harts in a single core are controlled by the same DM. In most platforms there will only be one DM that controls all the harts in the platform.

DMs provide run control of their harts in the platform. Abstract commands provide access to GPRs. Additional registers are accessible through abstract commands or by writing programs to the optional Program Buffer.

The Program Buffer allows the debugger to execute arbitrary instructions on a hart. This mechanism can also be used to access memory. An optional system bus access block allows memory accesses without using a RISC-V hart to perform the access.

Each RISC-V hart may implement a Trigger Module. When trigger conditions are met, harts will halt and inform the debug module that they have halted.

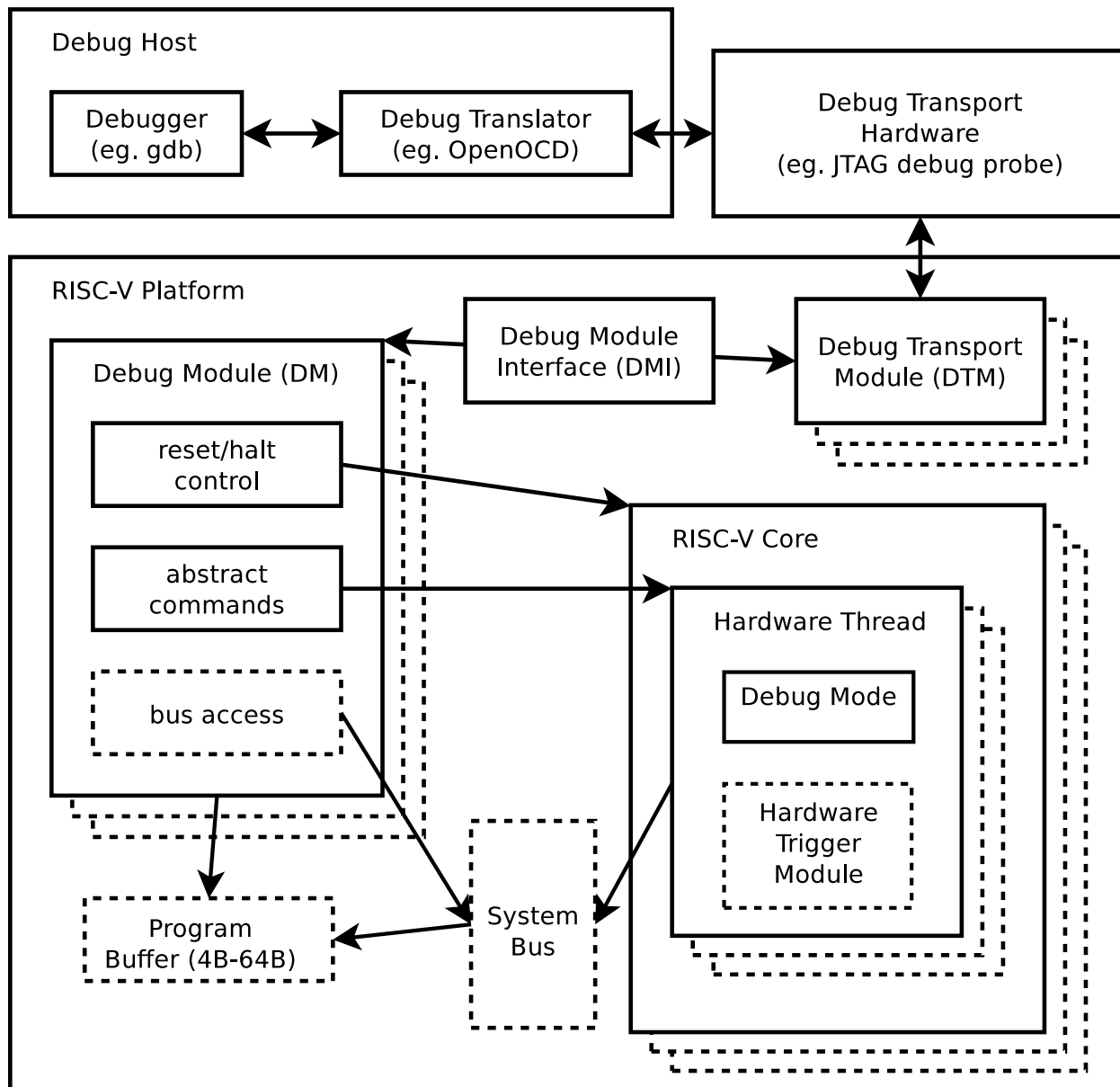


Figure 2.1: RISC-V Debug System Overview

Chapter 3

Debug Module (DM)

The Debug Module implements a translation interface between abstract debug operations and their specific implementation. It might support the following operations:

1. Give the debugger necessary information about the implementation. (Required)
2. Allow any individual hart to be halted and resumed. (Required)
3. Provide status on which harts are halted. (Required)
4. Provide abstract read and write access to a halted hart’s GPRs. (Required)
5. Provide access to a reset signal that allows debugging from the very first instruction after reset. (Required)
6. Provide a mechanism to allow debugging harts immediately out of reset (regardless of the reset cause). (Optional)
7. Provide abstract access to non-GPR hart registers. (Optional)
8. Provide a Program Buffer to force the hart to execute arbitrary instructions. (Optional)
9. Allow multiple harts to be halted, resumed, and/or reset at the same time. (Optional)
10. Allow memory access from a hart’s point of view. (Optional)
11. Allow direct System Bus Access. (Optional)

In order to be compliant with this specification an implementation must:

1. Implement all the required features listed above.
2. Implement at least one of Program Buffer, System Bus Access, or Abstract Access Memory command mechanisms.
3. Do at least one of:
 - (a) Implement the Program Buffer.
 - (b) Implement abstract access to all registers that are visible to software running on the hart including all the registers that are present on the hart and listed in Table 3.3.
 - (c) Implement abstract access to at least all GPRs, `dcsr`, and `dpc`, and advertise the implementation as conforming to the “Minimal RISC-V Debug Specification 0.13.2”, instead of the “RISC-V Debug Specification 0.13.2”.

A single DM can debug up to 2^{20} harts.

3.1 Debug Module Interface (DMI)

Debug Modules are slaves to a bus called the Debug Module Interface (DMI). The master of the bus is the **Debug Transport Module(s)**. The Debug Module Interface can be a trivial bus with **one master and one slave**, or use a more full-featured bus like TileLink or the **AMBA** Advanced Peripheral Bus. The details are left to the system designer.

The **DMI uses between 7 and 32 address bits**. It supports read and write operations. The bottom of the address space is used for the first (and usually only) DM. Extra space can be used for custom debug devices, other cores, additional DMs, etc. If there are additional DMs on this DMI, the base address of the next DM in the DMI address space is given in **nextdm**.

The Debug Module is controlled **via register accesses** to its DMI address space.

3.2 Reset Control

The Debug Module controls a global reset signal, **ndmreset** (non-debug module reset), which can reset, or hold in reset, every component in the platform, except for the Debug Module and Debug Transport Modules. Exactly what is affected by this reset is implementation dependent, as long as it is possible to debug programs from the first instruction executed. The Debug Module's own state and registers should only be reset at power-up and while **dmactive** in **dmcontrol** is 0. The halt state of harts should be maintained across system reset provided that **dmactive** is 1, although trigger CSRs may be cleared.

Due to clock and power domain crossing issues, it may not be possible to perform arbitrary DMI accesses across system reset. While **ndmreset** or any external reset is asserted, the only supported DM operation is accessing **dmcontrol**. The behavior of other accesses is undefined.

There is no requirement on the duration of the assertion of **ndmreset**. The implementation must ensure that a write of **ndmreset** to 1 followed by a write of **ndmreset** to 0 triggers system reset. The system may take an arbitrarily long time to come out of reset, as reported by **allunavail**, **anyunavail**.

Individual harts (or several at once) can be reset by selecting them, setting and then clearing **hartreset**. In this case an implementation may reset more harts than just the ones that are selected. The debugger can discover which other harts are reset (if any) by selecting them and checking **anyhavereset** and **allhavereset**.

When harts have been reset, they must set a sticky **havereset** state bit. The conceptual **havereset** state bits can be read for selected harts in **anyhavereset** and **allhavereset** in **dmstatus**. These bits must be set regardless of the cause of the reset. The **havereset** bits for the selected harts can be cleared by writing 1 to **ackhavereset** in **dmcontrol**. The **havereset** bits may or may not be cleared when **dmactive** is low.

When a hart comes out of reset and **haltreq** or **resethaltreq** are set, the hart will immediately enter Debug Mode. Otherwise it will execute normally.

3.3 Selecting Harts

Up to 2^{20} harts can be connected to a single DM. The debugger selects a hart, and then subsequent halt, resume, reset, and debugging commands are specific to that hart.

To enumerate all the harts, a debugger must first determine `HARTSELLEN` by writing all ones to `hartsel` (assuming the maximum size) and reading back the value to see which bits were actually set. Then it selects each hart starting from 0 until either `anynonexistent` in `dmstatus` is 1, or the highest index (depending on `HARTSELLEN`) is reached.

The debugger can discover the mapping between hart indices and `mhartid` by using the interface to read `mhartid`, or by reading the system's configuration string.

3.3.1 Selecting a Single Hart

All debug modules must support selecting a single hart. The debugger can select a hart by writing its index to `hartsel`. Hart indexes start at 0 and are contiguous until the final index.

3.3.2 Selecting Multiple Harts

Debug Modules may implement a Hart Array Mask register to allow selecting multiple harts at once. The n th bit in the Hart Array Mask register applies to the hart with index n . If the bit is 1 then the hart is selected. Usually a DM will have a Hart Array Mask register exactly wide enough to select all the harts it supports, but it's allowed to tie any of these bits to 0.

The debugger can set bits in the hart array mask register using `hawindowssel` and `hawindow`, then apply actions to all selected harts by setting `hasel`. If this feature is supported, multiple harts can be halted, resumed, and reset simultaneously. The state of the hart array mask register is not affected by setting or clearing `hasel`.

Only the actions initiated by `dmcontrol` can apply to multiple harts at once, Abstract Commands apply only to the hart selected by `hartsel`.

3.4 Hart States

Every hart that can be selected is in exactly one of four states. Which state the selected harts are in is reflected by `allnonexistent`, `anynonexistent`, `allunavail`, `anyunavail`, `allrunning`, `anyrunning`, `allhalted`, and `anyhalted`.

Harts are nonexistent if they will never be part of this system, no matter how long a user waits. E.g. in a simple single-hart system only one hart exists, and all others are nonexistent. Debuggers may assume that a system has no harts with indexes higher than the first nonexistent one.

Harts are unavailable if they might exist/become available at a later time, or if there are other harts with higher indexes than this one. Harts may be unavailable for a variety of reasons including being

reset, temporarily powered down, and not being plugged into the system. Systems with very large number of harts may permanently disable some during manufacturing, leaving holes in the otherwise continuous hart index space. In order to let the debugger discover all harts, they must show up as unavailable even if there is no chance of them ever becoming available.

Harts are running when they are executing normally, as if no debugger was attached. This includes being in a low power mode or waiting for an interrupt, as long as a halt request will result in the hart being halted.

Harts are halted when they are in Debug Mode, only performing tasks on behalf of the debugger.

Which states a hart that is reset goes through is implementation dependent. Harts may be unavailable while reset is asserted, and some time after reset is deasserted. They might transition to running for some time after reset is deasserted. Finally they end up either running or halted, depending on [haltreq](#) and [resethaltreq](#).

3.5 Run Control

For every hart, the Debug Module tracks 4 conceptual bits of state: halt request, resume ack, halt-on-reset request, and hart reset. (The hart reset and halt-on-reset request bits are optional.) These 4 bits reset to 0, except for resume ack, which may reset to either 0 or 1. The DM receives halted, running, and havereset signals from each hart. The debugger can observe the state of resume ack in [allresumeack](#) and [anyresumeack](#), and the state of halted, running, and havereset signals in [allhalted](#), [anyhalted](#), [allrunning](#), [anyrunning](#), [allhavereset](#), and [anyhavereset](#). The state of the other bits cannot be observed directly.

When a debugger writes 1 to [haltreq](#), each selected hart's halt request bit is set. When a running hart, or a hart just coming out of reset, sees its halt request bit high, it responds by halting, deasserting its running signal, and asserting its halted signal. Halted harts ignore their halt request bit.

When a debugger writes 1 to [resumereq](#), each selected hart's resume ack bit is cleared and each selected, halted hart is sent a resume request. Harts respond by resuming, clearing their halted signal, and asserting their running signal. At the end of this process the resume ack bit is set. These status signals of all selected harts are reflected in [allresumeack](#), [anyresumeack](#), [allrunning](#), and [anyrunning](#). Resume requests are ignored by running harts.

When halt or resume is requested, a hart must respond in less than one second, unless it is unavailable. (How this is implemented is not further specified. A few clock cycles will be a more typical latency).

The DM can implement optional halt-on-reset bits for each hart, which it indicates by setting [hasresethaltreq](#) to 1. This means the DM implements the [setresethaltreq](#) and [clrresethaltreq](#) bits. Writing 1 to [setresethaltreq](#) sets the halt-on-reset request bit for each selected hart. When a hart's halt-on-reset request bit is set, the hart will immediately enter debug mode on the next deassertion of its reset. This is true regardless of the reset's cause. The hart's halt-on-reset request bit remains set until cleared by the debugger writing 1 to [clrresethaltreq](#) while the hart is selected, or by DM reset.

3.6 Abstract Commands

The DM supports a set of abstract commands, most of which are optional. Depending on the implementation, the debugger may be able to perform some abstract commands even when the selected hart is not halted. Debuggers can only determine which abstract commands are supported by a given hart in a given state by attempting them and then looking at `cmderr` in `abstractcs` to see if they were successful. Commands may be supported with some options set, but not with other options set. If a command has unsupported options set, the DM must set `cmderr` to 2 (not supported).

Example: Every system must support the Access Register command, but may not support accessing CSRs. If the debugger requests to read a CSR in that case, the command will return “not supported.”

Debuggers execute abstract commands by writing them to `command`. They can determine whether an abstract command is complete by reading `busy` in `abstractcs`. After completion, `cmderr` indicates whether the command was successful or not. Commands may fail because a hart is not halted, not running, unavailable, or because they encounter an error during execution.

If the command takes arguments, the debugger must write them to the `data` registers before writing to `command`. If a command returns results, the Debug Module must ensure they are placed in the `data` registers before `busy` is cleared. Which `data` registers are used for the arguments is described in Table 3.1. In all cases the least-significant word is placed in the lowest-numbered `data` register. The argument width depends on the command being executed, and is `DXLEN` where not explicitly specified.

Table 3.1: Use of Data Registers

Argument Width	arg0/return value	arg1	arg2
32	<code>data0</code>	<code>data1</code>	<code>data2</code>
64	<code>data0</code> , <code>data1</code>	<code>data2</code> , <code>data3</code>	<code>data4</code> , <code>data5</code>
128	<code>data0</code> – <code>data3</code>	<code>data4</code> – <code>data7</code>	<code>data8</code> – <code>data11</code>

The Abstract Command interface is designed to allow a debugger to write commands as fast as possible, and then later check whether they completed without error. In the common case the debugger will be much slower than the target and commands succeed, which allows for maximum throughput. If there is a failure, the interface ensures that no commands execute after the failing one. To discover which command failed, the debugger has to look at the state of the DM (e.g. contents of `data0`) or hart (e.g. contents of a register modified by a Program Buffer program) to determine which one failed.

Before starting an abstract command, a debugger must ensure that `haltreq`, `resumereq`, and `ackhavereset` are all 0.

While an abstract command is executing (`busy` in `abstractcs` is high), a debugger must not change `hartsel`, and must not write 1 to `haltreq`, `resumereq`, `ackhavereset`, `setresethaltreq`, or `clrresethaltreq`.

If an abstract command does not complete in the expected time and appears to be hung, the following procedure can be attempted to abort the command: First the debugger resets the hart (using `hartreset` or `ndmreset`), and then it resets the Debug Module (using `dmactive`).

If an abstract command is started while the selected hart is unavailable or if a hart becomes unavailable while executing an abstract command, then the Debug Module may terminate the abstract command, setting `busy` low, and `cmderr` to 4 (halt/resume). Alternatively, the command could just appear to be hung (`busy` never goes low).

3.6.1 Abstract Command Listing

This section describes each of the different abstract commands and how their fields should be interpreted when they are written to `command`.

Each abstract command is a 32-bit value. The top 8 bits contain `cmdtype` which determines the kind of command. Table 3.2 lists all commands.

Table 3.2: Meaning of `cmdtype`

<code>cmdtype</code>	Command	Page
0	Access Register Command	12
1	Quick Access	14
2	Access Memory Command	14

3.6.1.1 Access Register

This command gives the debugger access to CPU registers and allows it to execute the Program Buffer. It performs the following sequence of operations:

1. If `write` is clear and `transfer` is set, then copy data from the register specified by `regno` into the `arg0` region of `data`, and perform any side effects that occur when this register is read from M-mode.
2. If `write` is set and `transfer` is set, then copy data from the `arg0` region of `data` into the register specified by `regno`, and perform any side effects that occur when this register is written from M-mode.
3. If `aarpostincrement` is set, increment `regno`.
4. Execute the Program Buffer, if `postexec` is set.

If any of these operations fail, `cmderr` is set and none of the remaining steps are executed. An implementation may detect an upcoming failure early, and fail the overall command before it reaches the step that would cause failure. If the failure is that the requested register does not exist in the hart, `cmderr` must be set to 3 (exception).

Debug Modules must implement this command and must support read and write access to all GPRs when the selected hart is halted. Debug Modules may optionally support accessing other registers, or accessing registers when the hart is running. Each individual register (aside from GPRs) may be supported differently across read, write, and halt status.

The encoding of `aarsize` was chosen to match `sbaccess` in `sbc`.

This command modifies `arg0` only when a register is read. The other `data` registers are not changed.

Table 3.3: Abstract Register Numbers

0x0000 – 0x0fff	CSRs. The “PC” can be accessed here through dpc .
0x1000 – 0x101f	GPRs
0x1020 – 0x103f	Floating point registers
0xc000 – 0xffff	Reserved for non-standard extensions and internal use.

31	24	23	22	20	19	18	17	16	15	0
cmdtype	0	aarsize	aarpostincrement	postexec	transfer	write	regno			
8	1	3	1	1	1	1	1	16		

Field	Description
cmdtype	This is 0 to indicate Access Register Command.
aarsize	2: Access the lowest 32 bits of the register. 3: Access the lowest 64 bits of the register. 4: Access the lowest 128 bits of the register. If aarsize specifies a size larger than the register’s actual size, then the access must fail. If a register is accessible, then reads of aarsize less than or equal to the register’s actual size must be supported. This field controls the Argument Width as referenced in Table 3.1.
aarpostincrement	0: No effect. This variant must be supported. 1: After a successful register access, regno is incremented (wrapping around to 0). Supporting this variant is optional.
postexec	0: No effect. This variant must be supported, and is the only supported one if progbuFSIZE is 0. 1: Execute the program in the Program Buffer exactly once after performing the transfer, if any. Supporting this variant is optional.
transfer	0: Don’t do the operation specified by write . 1: Do the operation specified by write . This bit can be used to just execute the Program Buffer without having to worry about placing valid values into aarsize or regno .
write	When transfer is set: 0: Copy data from the specified register into arg0 portion of data . 1: Copy data from arg0 portion of data into the specified register.
regno	Number of the register to access, as described in Table 3.3. dpc may be used as an alias for PC if this command is supported on a non-halted hart.

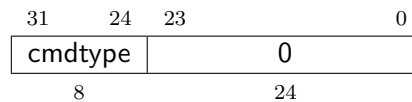
3.6.1.2 Quick Access

Perform the following sequence of operations:

1. If the hart is halted, the command sets `cmderr` to “halt/resume” and does not continue.
2. Halt the hart. If the hart halts for some other reason (e.g. breakpoint), the command sets `cmderr` to “halt/resume” and does not continue.
3. Execute the Program Buffer. If an exception occurs, `cmderr` is set to “exception” and the program buffer execution ends, but the quick access command continues.
4. Resume the hart.

Implementing this command is optional.

This command does not touch the `data` registers.



Field	Description
cmdtype	This is 1 to indicate Quick Access command.

3.6.1.3 Access Memory

This command lets the debugger perform memory accesses, with the exact same memory view and permissions as the selected hart has. This includes access to hart-local memory-mapped registers, etc. The command performs the following sequence of operations:

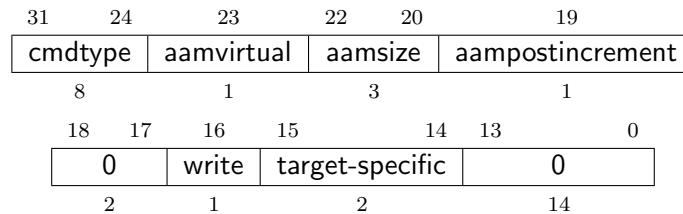
1. Copy data from the memory location specified in `arg1` into the `arg0` portion of `data`, if `write` is clear.
2. Copy data from the `arg0` portion of `data` into the memory location specified in `arg1`, if `write` is set.
3. If `aampostincrement` is set, increment `arg1`.

If any of these operations fail, `cmderr` is set and none of the remaining steps are executed. An access may only fail if the hart, running M-mode code, might encounter that same failure when it attempts the same access. An implementation may detect an upcoming failure early, and fail the overall command before it reaches the step that would cause failure.

Debug Modules may optionally implement this command and may support read and write access to memory locations when the selected hart is running or halted. If this command supports memory accesses while the hart is running, it must also support memory accesses while the hart is halted.

The encoding of `aamsize` was chosen to match `sbaccess` in `sbc`s.

This command modifies `arg0` only when memory is read. It modifies `arg1` only if `aampostincrement` is set. The other `data` registers are not changed.



Field	Description
<code>cmdtype</code>	This is 2 to indicate Access Memory Command.
<code>aamvirtual</code>	An implementation does not have to implement both virtual and physical accesses, but it must fail accesses that it doesn't support. 0: Addresses are physical (to the hart they are performed on). 1: Addresses are virtual, and translated the way they would be from M-mode, with <code>MPRV</code> set.
<code>aamsize</code>	0: Access the lowest 8 bits of the memory location. 1: Access the lowest 16 bits of the memory location. 2: Access the lowest 32 bits of the memory location. 3: Access the lowest 64 bits of the memory location. 4: Access the lowest 128 bits of the memory location.
<code>aampostincrement</code>	After a memory access has completed, if this bit is 1, increment <code>arg1</code> (which contains the address used) by the number of bytes encoded in <code>aamsize</code> .
<code>write</code>	0: Copy data from the memory location specified in <code>arg1</code> into <code>arg0</code> portion of <code>data</code> . 1: Copy data from <code>arg0</code> portion of <code>data</code> into the memory location specified in <code>arg1</code> .
<code>target-specific</code>	These bits are reserved for target-specific uses.

3.7 Program Buffer

To support executing arbitrary instructions on a halted hart, a Debug Module can include a Program Buffer that a debugger can write small programs to. Systems that support all necessary functionality using abstract commands only may choose to omit the Program Buffer.

A debugger can write a small program to the Program Buffer, and then execute it exactly once with the Access Register Abstract Command, setting the `postexec` bit in `command`. The debugger can write whatever program it likes (including jumps out of the Program Buffer), but the program must end with `ebreak` or `c.ebreak`. An implementation may support an implied `ebreak` that is executed when a hart runs off the end of the Program Buffer. This is indicated by `impebreak`. With this feature, a Program Buffer of just 2 32-bit words can offer efficient debugging.

If `progbuFSIZE` is 1, `impebreak` must be 1. It is possible that the Program Buffer can hold only one 32- or 16-bit instruction, so the debugger must only write a single instruction in this case, regardless of its size. This instruction can be a 32-bit instruction, or a compressed instruction in the lower 16 bits accompanied by a compressed `nop` in the upper 16 bits.

The slightly inconsistent behavior with a Program Buffer of size 1 is to accommodate hardware designs that prefer to stuff instructions directly into the pipeline when halted, instead of having the Program Buffer exist in the address space somewhere.

While these programs are executed, the hart does not leave Debug Mode (see Section 4.1). If an exception is encountered during execution of the Program Buffer, no more instructions are executed, the hart remains in Debug Mode, and `cmderr` is set to 3 (exception error). If the debugger executes a program that doesn't terminate with an `ebreak` instruction, the hart will remain in Debug Mode and the debugger will lose control of the hart.

Executing the Program Buffer may clobber `dpc`. If that is the case, it must be possible to read/write `dpc` using an abstract command with `postexec` not set. The debugger must attempt to save `dpc` between halting and executing a Program Buffer, and then restore `dpc` before leaving Debug Mode.

Allowing Program Buffer execution to clobber `dpc` allows for direct implementations that don't have a separate PC register, and do need to use the PC when executing the Program Buffer.

The Program Buffer may be implemented as RAM which is accessible to the hart. A debugger can determine if this is the case by executing small programs that attempt to write and read back relative to `pc` while executing from the Program Buffer. If so, the debugger has more flexibility in what it can do with the program buffer.

3.8 Overview of States

Figure 3.1 shows a conceptual view of the states passed through by a hart during run/halt debugging as influenced by the different fields of `dmcontrol`, `abstractcs`, `abstractauto`, and `command`.

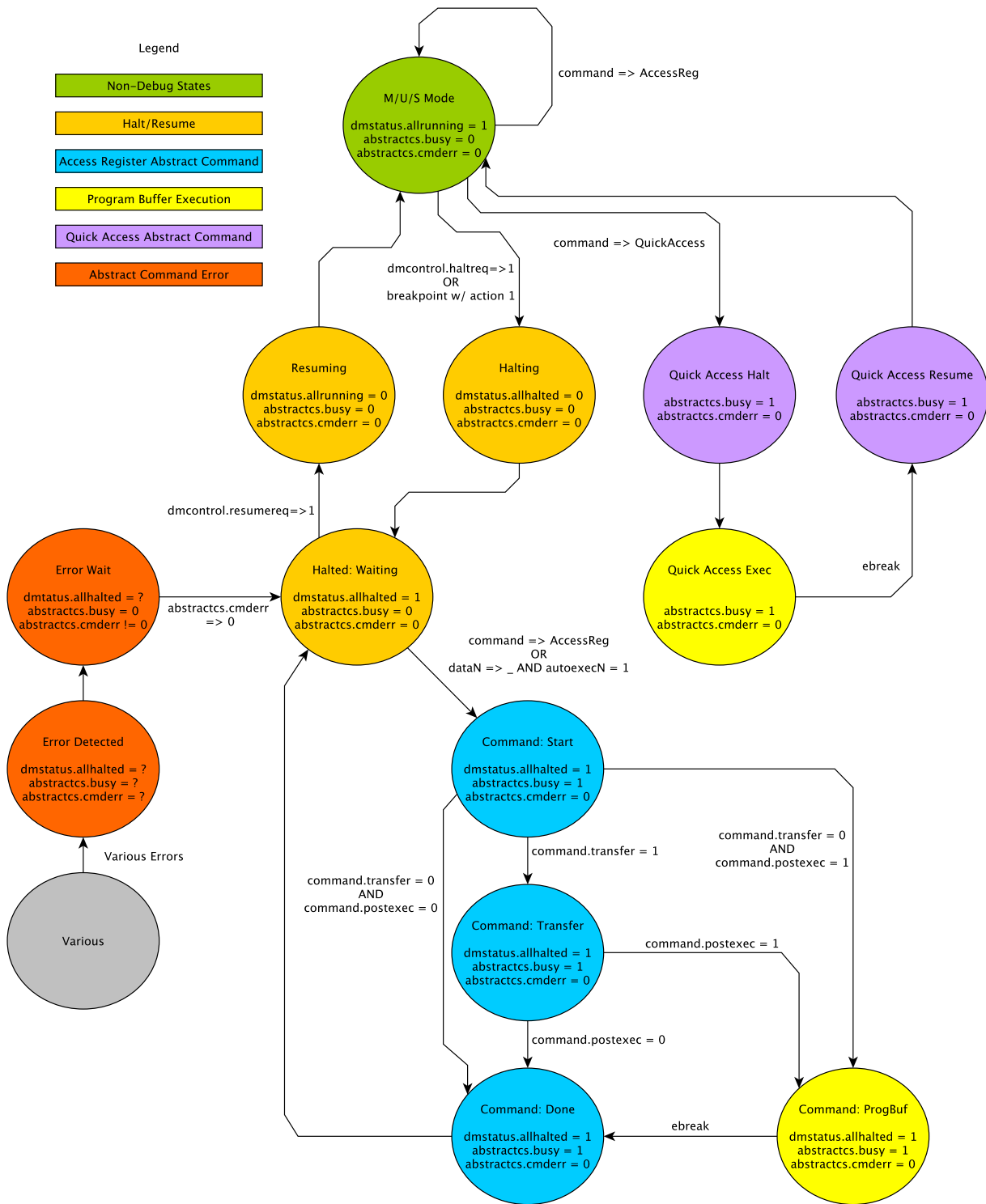


Figure 3.1: Run/Halt Debug State Machine for single-hart systems. As only a small amount of state is visible to the debugger, the states and transitions are conceptual.

3.9 System Bus Access

A debugger can access memory from a hart’s point of view using a Program Buffer or the Abstract Access Memory command. (Both these features are optional.) A Debug Module may also include a System Bus Access block to provide memory access without involving a hart, regardless of whether Program Buffer is implemented. The System Bus Access block uses physical addresses.

The System Bus Access block may support 8-, 16-, 32-, 64-, and 128-bit accesses. Table 3.7 shows which bits in `sbddata` are used for each access size.

Table 3.7: System Bus Data Bits

Access Size	Data Bits
8	<code>sbddata0</code> bits 7:0
16	<code>sbddata0</code> bits 15:0
32	<code>sbddata0</code>
64	<code>sbddata1</code> , <code>sbddata0</code>
128	<code>sbddata3</code> , <code>sbddata2</code> , <code>sbddata1</code> , <code>sbddata0</code>

Depending on the microarchitecture, data accessed through System Bus Access may not always be coherent with that observed by each hart. It is up to the debugger to enforce coherency if the implementation does not. This specification does not define a standard way to do this. Possibilities may include writing to special memory-mapped locations, or executing special instructions via the Program Buffer.

Implementing a System Bus Access block has several benefits even when a Debug Module also implements a Program Buffer. First, it is possible to access memory in a running system with minimal impact. Second, it may improve performance when accessing memory. Third, it may provide access to devices that a hart does not have access to.

3.10 Minimally Intrusive Debugging

Depending on the task it is performing, some harts can only be halted very briefly. There are several mechanisms that allow accessing resources in such a running system with a minimal impact on the running hart.

First, an implementation may allow some abstract commands to execute without halting the hart.

Second, the Quick Access abstract command can be used to halt a hart, quickly execute the contents of the Program Buffer, and let the hart run again. Combined with instructions that allow Program Buffer code to access the `data` registers, as described in 3.12.3, this can be used to quickly perform a memory or register access. For some systems this will be too intrusive, but many systems that can’t be halted can bear an occasional hiccup of a hundred or less cycles.

Third, if the System Bus Access block is implemented, it can be used while a hart is running to access system memory.

3.11 Security

To protect intellectual property it may be desirable to lock access to the Debug Module. To allow access during a manufacturing process and not afterwards, a reasonable solution could be to add a fuse bit to the Debug Module that can be used to be permanently disable it. Since this is technology specific, it is not further addressed in this spec.

Another option is to allow the DM to be unlocked only by users who have an access key. Between `authenticated`, `authbusy`, and `authdata` arbitrarily complex authentication mechanism can be supported. When `authenticated` is clear, the DM must not interact with the rest of the platform, nor expose details about the harts connected to the DM. All DM registers should read 0, while writes should be ignored, with the following mandatory exceptions:

1. `authenticated` in `dmstatus` is readable.
2. `authbusy` in `dmstatus` is readable.
3. `version` in `dmstatus` is readable.
4. `dmactive` in `dmcontrol` is readable and writable.
5. `authdata` is readable and writable.

3.12 Debug Module Registers

The registers described in this section are accessed **over the DMI bus**. Each DM has a base address (which is 0 for the first DM). The register addresses below are **offsets from this base address**.

When read, unimplemented Debug Module DMI Registers return 0. Writing them has no effect.

For each register it is possible to determine that it is implemented by reading it and getting a non-zero value (e.g. `sbc`), or by checking bits in another register (e.g. `progbuysize`).

Table 3.8: Debug Module Debug Bus Registers

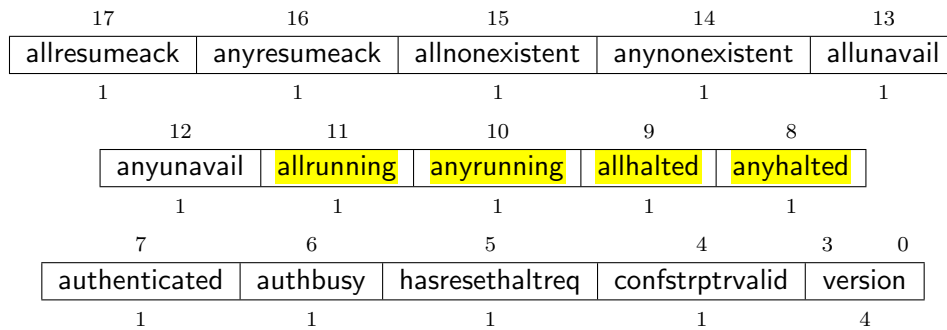
Address	Name	Page
0x04	Abstract Data 0 (data0)	30
0x0f	Abstract Data 11 (data11)	
0x10	Debug Module Control (dmcontrol)	22
0x11	Debug Module Status (dmstatus)	20
0x12	Hart Info (hartinfo)	25
0x13	Halt Summary 1 (haltsum1)	31
0x14	Hart Array Window Select (hawindowse1)	26
0x15	Hart Array Window (hawindow)	26
0x16	Abstract Control and Status (abstractcs)	27
0x17	Abstract Command (command)	28
0x18	Abstract Command Autoexec (abstractauto)	29
0x19	Configuration String Pointer 0 (confstrptr0)	29
0x1a	Configuration String Pointer 1 (confstrptr1)	
0x1b	Configuration String Pointer 2 (confstrptr2)	
0x1c	Configuration String Pointer 3 (confstrptr3)	
0x1d	Next Debug Module (nextdm)	30
0x20	Program Buffer 0 (progbuf0)	30
0x2f	Program Buffer 15 (progbuf15)	
0x30	Authentication Data (authdata)	31
0x34	Halt Summary 2 (haltsum2)	32
0x35	Halt Summary 3 (haltsum3)	32
0x37	System Bus Address 127:96 (sbaddress3)	36
0x38	System Bus Access Control and Status (sbc s)	32
0x39	System Bus Address 31:0 (sbaddress0)	34
0x3a	System Bus Address 63:32 (sbaddress1)	35
0x3b	System Bus Address 95:64 (sbaddress2)	35
0x3c	System Bus Data 31:0 (sbd ata0)	36
0x3d	System Bus Data 63:32 (sbd ata1)	37
0x3e	System Bus Data 95:64 (sbd ata2)	37
0x3f	System Bus Data 127:96 (sbd ata3)	38
0x40	Halt Summary 0 (haltsum0)	31

3.12.1 Debug Module **Status** (**dmstatus**, at **0x11**)

This register reports status for the overall Debug Module as well as the currently selected harts, as defined in [hasel](#). Its address will not change in the future, because it contains [version](#).

This entire register is **read-only**.

31	23	22	21	20	19	18
0	impebreak	0	allhavereset	anyhavereset		
9	1	2	1	1		



Field	Description	Access	Reset
impebreak	If 1, then there is an implicit ebreak instruction at the non-existent word immediately after the Program Buffer. This saves the debugger from having to write the ebreak itself, and allows the Program Buffer to be one word smaller. This must be 1 when progbuFSIZE is 1.	R	Preset
allhavereset	This field is 1 when all currently selected harts have been reset and reset has not been acknowledged for any of them.	R	-
anyhavereset	This field is 1 when at least one currently selected hart has been reset and reset has not been acknowledged for that hart.	R	-
allresumeack	This field is 1 when all currently selected harts have acknowledged their last resume request.	R	-
anyresumeack	This field is 1 when any currently selected hart has acknowledged its last resume request.	R	-
allnonexistent	This field is 1 when all currently selected harts do not exist in this platform.	R	-
anynonexistent	This field is 1 when any currently selected hart does not exist in this platform.	R	-
allunavail	This field is 1 when all currently selected harts are unavailable.	R	-
anyunavail	This field is 1 when any currently selected hart is unavailable.	R	-
allrunning	This field is 1 when all currently selected harts are running.	R	-
anyrunning	This field is 1 when any currently selected hart is running.	R	-
allhalted	This field is 1 when all currently selected harts are halted.	R	-
anyhalted	This field is 1 when any currently selected hart is halted.	R	-

Continued on next page

Field	Description	Access	Reset
authenticated	0: Authentication is required before using the DM. 1: The authentication check has passed. On components that don't implement authentication, this bit must be preset as 1.	R	Preset
authbusy	0: The authentication module is ready to process the next read/write to authdata . 1: The authentication module is busy. Accessing authdata results in unspecified behavior. authbusy only becomes set in immediate response to an access to authdata .	R	0
hasresethaltreq	1 if this Debug Module supports halt-on-reset functionality controllable by the setresethaltreq and clrresethaltreq bits. 0 otherwise.	R	Preset
confstrptrvalid	0: confstrptr0 – confstrptr3 hold information which is not relevant to the configuration string. 1: confstrptr0 – confstrptr3 hold the address of the configuration string.	R	Preset
version	0: There is no Debug Module present. 1: There is a Debug Module and it conforms to version 0.11 of this specification. 2: There is a Debug Module and it conforms to version 0.13 of this specification. 15: There is a Debug Module but it does not conform to any available version of this spec.	R	2

3.12.2 Debug Module [Control](#) ([dmcontrol](#), at [0x10](#))

This register [controls](#) the overall Debug Module as well as the currently selected harts, as defined in [hasel](#).

Throughout this document we refer to [hartsel](#), which is [hartselhi](#) combined with [hartsello](#). While the spec allows for 20 [hartsel](#) bits, an implementation may choose to implement fewer than that. The actual width of [hartsel](#) is called `HARTSELLEN`. It must be at least 0 and at most 20. A debugger should discover `HARTSELLEN` by writing all ones to [hartsel](#) (assuming the maximum size) and reading back the value to see which bits were actually set. Debuggers must not change [hartsel](#) while an abstract command is executing.

There are separate [setresethaltreq](#) and [clrresethaltreq](#) bits so that it is possible to write [dmcontrol](#) without changing the halt-on-reset request bit for each selected hart, when not all selected harts have the same configuration.

On any given write, a debugger may only write 1 to at most one of the following bits: [resumereq](#), [hartreset](#), [ackhavereset](#), [setresethaltreq](#), and [clrresethaltreq](#). The others must be written 0.

`resethaltreq` is an optional internal bit of per-hart state that cannot be read, but can be written with `setresethaltreq` and `clrresethaltreq`.



Field	Description	Access	Reset
haltreq	Writing 0 clears the halt request bit for all currently selected harts. This may cancel outstanding halt requests for those harts. Writing 1 sets the halt request bit for all currently selected harts. Running harts will halt whenever their halt request bit is set. Writes apply to the new value of <code>hartsel</code> and <code>hasel</code>.	W	-
resumereq	Writing 1 causes the currently selected harts to resume once, if they are halted when the write occurs. It also clears the resume ack bit for those harts. <code>resumereq</code> is ignored if <code>haltreq</code> is set. Writes apply to the new value of <code>hartsel</code> and <code>hasel</code>.	W1	-
hartreset	This optional field writes the reset bit for all the currently selected harts. To perform a reset the debugger writes 1, and then writes 0 to deassert the reset signal. While this bit is 1, the debugger must not change which harts are selected. If this feature is not implemented, the bit always stays 0, so after writing 1 the debugger can read the register back to see if the feature is supported. Writes apply to the new value of <code>hartsel</code> and <code>hasel</code>.	R/W	0
ackhavereset	0: No effect. 1: Clears <code>havereset</code> for any selected harts. Writes apply to the new value of <code>hartsel</code> and <code>hasel</code>.	W1	-

Continued on next page

Field	Description	Access	Reset
hasel	Selects the definition of currently selected harts. 0: There is a single currently selected hart, that is selected by hartsel . 1: There may be multiple currently selected harts – the hart selected by hartsel , plus those selected by the hart array mask register. An implementation which does not implement the hart array mask register must tie this field to 0. A debugger which wishes to use the hart array mask register feature should set this bit and read back to see if the functionality is supported.	R/W	0
hartsello	The low 10 bits of hartsel : the DM-specific index of the hart to select. This hart is always part of the currently selected harts.	R/W	0
hartselhi	The high 10 bits of hartsel : the DM-specific index of the hart to select. This hart is always part of the currently selected harts.	R/W	0
setresethaltreq	This optional field writes the halt-on-reset request bit for all currently selected harts, unless clrresethaltreq is simultaneously set to 1. When set to 1, each selected hart will halt upon the next deassertion of its reset. The halt-on-reset request bit is not automatically cleared. The debugger must write to clrresethaltreq to clear it. Writes apply to the new value of hartsel and hasel . If hasresethaltreq is 0, this field is not implemented.	W1	-
clrresethaltreq	This optional field clears the halt-on-reset request bit for all currently selected harts. Writes apply to the new value of hartsel and hasel .	W1	-
ndmreset	This bit controls the reset signal from the DM to the rest of the system. The signal should reset every part of the system, including every hart, except for the DM and any logic required to access the DM. To perform a system reset the debugger writes 1, and then writes 0 to deassert the reset.	R/W	0

Continued on next page

Field	Description	Access	Reset
dmactive	This bit serves as a reset signal for the Debug Module itself. 0: The module's state, including authentication mechanism, takes its reset values (the dmactive bit is the only bit which can be written to something other than its reset value). 1: The module functions normally. No other mechanism should exist that may result in resetting the Debug Module after power up, with the possible (but not recommended) exception of a global reset signal that resets the entire platform. A debugger may pulse this bit low to get the Debug Module into a known state. Implementations may pay attention to this bit to further aid debugging, for example by preventing the Debug Module from being power gated while debugging is active.	R/W	0

3.12.3 Hart Info (hartinfo, at 0x12)

This register gives information about the hart currently selected by **hartsel**.

This register is optional. If it is not present it should read all-zero.

If this register is included, the debugger can do more with the Program Buffer by writing programs which explicitly access the **data** and/or **dscratch** registers.

This entire register is read-only.

31	24	23	20	19	17	16	15	12	11	0
0	nscratch	0	dataaccess	datasize	dataaddr					
8	4	3	1	4	12					

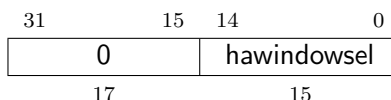
Field	Description	Access	Reset
nscratch	Number of dscratch registers available for the debugger to use during program buffer execution, starting from dscratch0 . The debugger can make no assumptions about the contents of these registers between commands.	R	Preset

Continued on next page

Field	Description	Access	Reset
dataaccess	0: The data registers are shadowed in the hart by CSRs. Each CSR is DXLEN bits in size, and corresponds to a single argument, per Table 3.1. 1: The data registers are shadowed in the hart's memory map. Each register takes up 4 bytes in the memory map.	R	Preset
datasize	If dataaccess is 0: Number of CSRs dedicated to shadowing the data registers. If dataaccess is 1: Number of 32-bit words in the memory map dedicated to shadowing the data registers. Since there are at most 12 data registers, the value in this register must be 12 or smaller.	R	Preset
dataaddr	If dataaccess is 0: The number of the first CSR dedicated to shadowing the data registers. If dataaccess is 1: Signed address of RAM where the data registers are shadowed, to be used to access relative to zero .	R	Preset

3.12.4 Hart Array Window Select ([hawindow](#)_{sel}, at 0x14)

This register selects which of the 32-bit portion of the hart array mask register (see Section 3.3.2) is accessible in [hawindow](#).

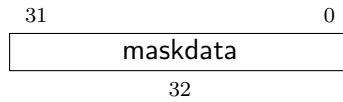


Field	Description	Access	Reset
hawindow _{sel}	The high bits of this field may be tied to 0, depending on how large the array mask register is. E.g. on a system with 48 harts only bit 0 of this field may actually be writable.	R/W	0

3.12.5 Hart Array Window ([hawindow](#), at 0x15)

This register provides R/W access to a 32-bit portion of the hart array mask register (see Section 3.3.2). The position of the window is determined by [hawindow_{sel}](#). I.e. bit 0 refers to hart [hawindow_{sel}](#) * 32, while bit 31 refers to hart [hawindow_{sel}](#) * 32 + 31.

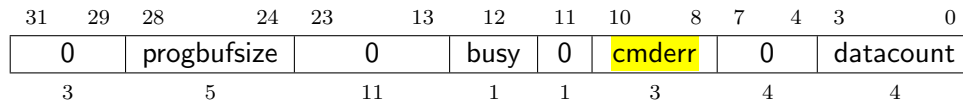
Since some bits in the hart array mask register may be constant 0, some bits in this register may be constant 0, depending on the current value of [hawindowssel](#).



3.12.6 Abstract Control and Status (abstractcs, at 0x16)

Writing this register while an abstract command is executing causes **cmderr** to be set to 1 (busy) if it is 0.

[datacount](#) must be at least 1 to support RV32 harts, 2 to support RV64 harts, or 4 to support RV128 harts.



Field	Description	Access	Reset
progbuysize	Size of the Program Buffer, in 32-bit words. Valid sizes are 0 - 16.	R	Preset
busy	1: An abstract command is currently being executed. This bit is set as soon as command is written, and is not cleared until that command has completed.	R	0

Continued on next page

Field	Description	Access	Reset
cmderr	Gets set if an abstract command fails. The bits in this field remain set until they are cleared by writing 1 to them. No abstract command is started until the value is reset to 0. This field only contains a valid value if busy is 0. 0 (none): No error. 1 (busy): An abstract command was executing while command, abstractcs, or abstractauto was written, or when one of the data or progbuf registers was read or written. This status is only written if cmderr contains 0. 2 (not supported): The requested command is not supported, regardless of whether the hart is running or not. 3 (exception): An exception occurred while executing the command (e.g. while executing the Program Buffer). 4 (halt/resume): The abstract command couldn't execute because the hart wasn't in the required state (running/halted), or unavailable. 5 (bus): The abstract command failed due to a bus error (e.g. alignment, access size, or timeout). 7 (other): The command failed for another reason.	R/W1C	0
datacount	Number of data registers that are implemented as part of the abstract command interface. Valid sizes are 1 – 12.	R	Preset

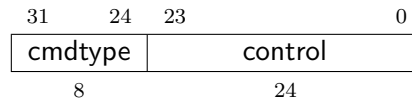
3.12.7 Abstract Command (command, at 0x17)

Writes to this register cause the corresponding abstract command to be executed.

Writing this register while an abstract command is executing causes **cmderr** to be set to 1 (busy) if it is 0.

If **cmderr** is non-zero, writes to this register are ignored.

***cmderr** inhibits starting a new command to accommodate debuggers that, for performance reasons, send several commands to be executed in a row without checking **cmderr** in between. They can safely do so and check **cmderr** at the end without worrying that one command failed but then a later command (which might have depended on the previous one succeeding) passed.*

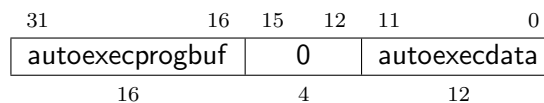


Field	Description	Access	Reset
cmdtype	The type determines the overall functionality of this abstract command.	W	0
control	This field is interpreted in a command-specific manner, described for each abstract command.	W	0

3.12.8 Abstract Command Autoexec (abstractauto, at 0x18)

This register is optional. Including it allows more efficient burst accesses. A debugger can detect whether it is support by setting bits and reading them back.

Writing this register while an abstract command is executing causes `cmderr` to be set to 1 (busy) if it is 0.



Field	Description	Access	Reset
autoexecprogbuf	When a bit in this field is 1, read or write accesses to the corresponding <code>progbuf</code> word cause the command in <code>command</code> to be executed again.	R/W	0
autoexecdata	When a bit in this field is 1, read or write accesses to the corresponding <code>data</code> word cause the command in <code>command</code> to be executed again.	R/W	0

3.12.9 Configuration String Pointer 0 (confstrptr0, at 0x19)

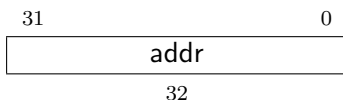
When `confstrptrvalid` is set, reading this register returns bits 31:0 of the configuration string pointer. Reading the other `confstrptr` registers returns the upper bits of the address.

When system bus mastering is implemented, this must be an address that can be used with the System Bus Access module. Otherwise, this must be an address that can be used to access the configuration string from the hart with ID 0.

If `confstrptrvalid` is 0, then the `confstrptr` registers hold identifier information which is not further specified in this document.

The configuration string itself is described in the Privileged Spec.

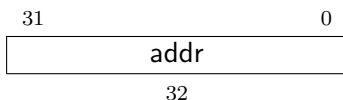
This entire register is read-only.



3.12.10 Next Debug Module (nextdm, at 0x1d)

If there is more than one DM accessible on this DMI, this register contains the base address of the next one in the chain, or 0 if this is the last one in the chain.

This entire register is read-only.



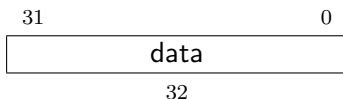
3.12.11 Abstract Data 0 (data0, at 0x04)

`data0` through `data11` are basic read/write registers that may be read or changed by abstract commands. `datacount` indicates how many of them are implemented, starting at `data0`, counting up. Table 3.1 shows how abstract commands use these registers.

Accessing these registers while an abstract command is executing causes `cmderr` to be set to 1 (busy) if it is 0.

Attempts to write them while `busy` is set does not change their value.

The values in these registers may not be preserved after an abstract command is executed. The only guarantees on their contents are the ones offered by the command in question. If the command fails, no assumptions can be made about the contents of these registers.

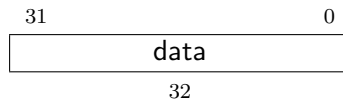


3.12.12 Program Buffer 0 (progbuf0, at 0x20)

`progbuf0` through `progbuf15` provide read/write access to the optional program buffer. `progbufsize` indicates how many of them are implemented starting at `progbuf0`, counting up.

Accessing these registers while an abstract command is executing causes `cmderr` to be set to 1 (busy) if it is 0.

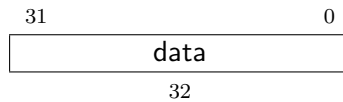
Attempts to write them while `busy` is set does not change their value.



3.12.13 Authentication Data (authdata, at 0x30)

This register serves as a 32-bit serial port to/from the authentication module.

When `authbusy` is clear, the debugger can communicate with the authentication module by reading or writing this register. There is no separate mechanism to signal overflow/underflow.

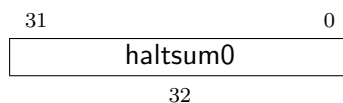


3.12.14 Halt Summary 0 (haltsum0, at 0x40)

Each bit in this read-only register indicates whether one specific hart is halted or not. Unavailable/nonexistent harts are not considered to be halted.

The LSB reflects the halt status of hart `{hartsel[19:5],5'h0}`, and the MSB reflects halt status of hart `{hartsel[19:5],5'h1f}`.

This entire register is read-only.



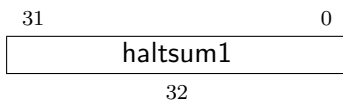
3.12.15 Halt Summary 1 (haltsum1, at 0x13)

Each bit in this read-only register indicates whether any of a group of harts is halted or not. Unavailable/nonexistent harts are not considered to be halted.

This register may not be present in systems with fewer than 33 harts.

The LSB reflects the halt status of harts `{hartsel[19:10],10'h0}` through `{hartsel[19:10],10'h1f}`. The MSB reflects the halt status of harts `{hartsel[19:10],10'h3e0}` through `{hartsel[19:10],10'h3ff}`.

This entire register is read-only.



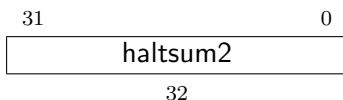
3.12.16 Halt Summary 2 (haltsum2, at 0x34)

Each bit in this read-only register indicates whether any of a group of harts is halted or not. Unavailable/nonexistent harts are not considered to be halted.

This register may not be present in systems with fewer than 1025 harts.

The LSB reflects the halt status of harts {hartsel[19:15],15'h0} through {hartsel[19:15],15'h3ff}. The MSB reflects the halt status of harts {hartsel[19:15],15'h7c00} through {hartsel[19:15],15'h7fff}.

This entire register is read-only.



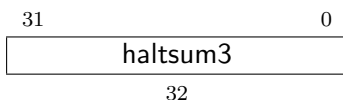
3.12.17 Halt Summary 3 (haltsum3, at 0x35)

Each bit in this read-only register indicates whether any of a group of harts is halted or not. Unavailable/nonexistent harts are not considered to be halted.

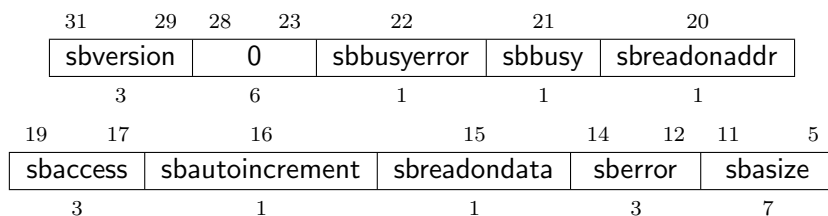
This register may not be present in systems with fewer than 32769 harts.

The LSB reflects the halt status of harts 20'h0 through 20'h7fff. The MSB reflects the halt status of harts 20'hf8000 through 20'hffff.

This entire register is read-only.



3.12.18 System Bus Access Control and Status (sbcs, at 0x38)



4	3	2	1	0
sbaccess128	sbaccess64	sbaccess32	sbaccess16	sbaccess8
1	1	1	1	1

Field	Description	Access	Reset
sbversion	0: The System Bus interface conforms to mainline drafts of this spec older than 1 January, 2018. 1: The System Bus interface conforms to this version of the spec. Other values are reserved for future versions.	R	1
sdbusyerror	Set when the debugger attempts to read data while a read is in progress, or when the debugger initiates a new access while one is already in progress (while sdbusy is set). It remains set until it's explicitly cleared by the debugger. While this field is set, no more system bus accesses can be initiated by the Debug Module.	R/WIC	0
sdbusy	When 1, indicates the system bus master is busy. (Whether the system bus itself is busy is related, but not the same thing.) This bit goes high immediately when a read or write is requested for any reason, and does not go low until the access is fully completed. Writes to sbus while sdbusy is high result in undefined behavior. A debugger must not write to sbus until it reads sdbusy as 0.	R	0
sbreadonaddr	When 1, every write to sbadress0 automatically triggers a system bus read at the new address.	R/W	0
sbaccess	Select the access size to use for system bus accesses. 0: 8-bit 1: 16-bit 2: 32-bit 3: 64-bit 4: 128-bit If sbaccess has an unsupported value when the DM starts a bus access, the access is not performed and serror is set to 4.	R/W	2
sbautoincrement	When 1, sbadress is incremented by the access size (in bytes) selected in sbaccess after every system bus access.	R/W	0
sbreadondata	When 1, every read from sbdatab0 automatically triggers a system bus read at the (possibly auto-incremented) address.	R/W	0

Continued on next page

Field	Description	Access	Reset
sberror	When the Debug Module's system bus master encounters an error, this field gets set. The bits in this field remain set until they are cleared by writing 1 to them. While this field is non-zero, no more system bus accesses can be initiated by the Debug Module. An implementation may report "Other" (7) for any error condition. 0: There was no bus error. 1: There was a timeout. 2: A bad address was accessed. 3: There was an alignment error. 4: An access of unsupported size was requested. 7: Other.	R/W1C	0
sbsize	Width of system bus addresses in bits. (0 indicates there is no bus access support.)	R	Preset
sbaccess128	1 when 128-bit system bus accesses are supported.	R	Preset
sbaccess64	1 when 64-bit system bus accesses are supported.	R	Preset
sbaccess32	1 when 32-bit system bus accesses are supported.	R	Preset
sbaccess16	1 when 16-bit system bus accesses are supported.	R	Preset
sbaccess8	1 when 8-bit system bus accesses are supported.	R	Preset

3.12.19 System Bus Address 31:0 (sbaddress0, at 0x39)

If **sbsize** is 0, then this register is not present.

When the system bus master is busy, writes to this register will set **sbbusyerror** and don't do anything else.

If **sberror** is 0, **sbbusyerror** is 0, and **sbreadonaddr** is set then writes to this register start the following:

1. Set **sbbusy**.
2. Perform a bus read from the new value of **sbaddress**.
3. If the read succeeded and **sbautoincrement** is set, increment **sbaddress**.
4. Clear **sbbusy**.

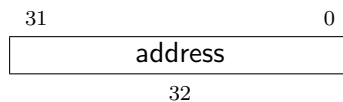


Field	Description	Access	Reset
address	Accesses bits 31:0 of the physical address in sbaddress .	R/W	0

3.12.20 System Bus Address 63:32 (sbaddress1, at 0x3a)

If **sbase** is less than 33, then this register is not present.

When the system bus master is busy, writes to this register will set `sbbusyerror` and don't do anything else.

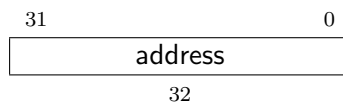


Field	Description	Access	Reset
address	Accesses bits 63:32 of the physical address in saddress (if the system address bus is that wide).	R/W	0

3.12.21 System Bus Address 95:64 (sbaddress2, at 0x3b)

If **sbase** is less than 65, then this register is not present.

When the system bus master is busy, writes to this register will set `sbbusyerror` and don't do anything else.

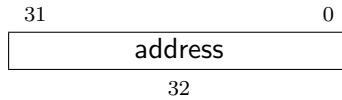


Field	Description	Access	Reset
address	Accesses bits 95:64 of the physical address in sbaddress (if the system address bus is that wide).	R/W	0

3.12.22 System Bus Address 127:96 (sbaddress3, at 0x37)

If **sbasize** is less than 97, then this register is not present.

When the system bus master is busy, writes to this register will set **sbbusyerror** and don't do anything else.



Field	Description	Access	Reset
address	Accesses bits 127:96 of the physical address in sbaddress (if the system address bus is that wide).	R/W	0

3.12.23 System Bus Data 31:0 (sbdata0, at 0x3c)

If all of the **sbaccess** bits in **sbc**s are 0, then this register is not present.

Any successful system bus read updates **sbdata**. If the width of the read access is less than the width of **sbdata**, the contents of the remaining high bits may take on any value.

If **sberror** or **sbbusyerror** both aren't 0 then accesses do nothing.

If the bus master is busy then accesses set **sbbusyerror**, and don't do anything else.

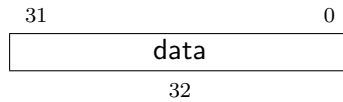
Writes to this register start the following:

1. Set **sbbusy**.
2. Perform a bus write of the new value of **sbdata** to **sbaddress**.
3. If the write succeeded and **sbautoincrement** is set, increment **sbaddress**.
4. Clear **sbbusy**.

Reads from this register start the following:

1. "Return" the data.
2. Set **sbbusy**.
3. If **sbreadondata** is set, perform a system bus read from the address contained in **sbaddress**, placing the result in **sbdata**.
4. If **sbautoincrement** is set, increment **sbaddress**.
5. Clear **sbbusy**.

Only `sbdata0` has this behavior. The other `sbdata` registers have no side effects. On systems that have buses wider than 32 bits, a debugger should access `sbdata0` after accessing the other `sbdata` registers.

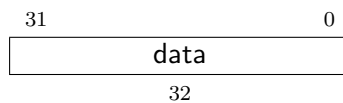


Field	Description	Access	Reset
data	Accesses bits 31:0 of <code>sbdata</code> .	R/W	0

3.12.24 System Bus Data 63:32 (`sbdata1`, at 0x3d)

If `sbaccess64` and `sbaccess128` are 0, then this register is not present.

If the bus master is busy then accesses set `sbbusyerror`, and don't do anything else.

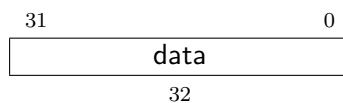


Field	Description	Access	Reset
data	Accesses bits 63:32 of <code>sbdata</code> (if the system bus is that wide).	R/W	0

3.12.25 System Bus Data 95:64 (`sbdata2`, at 0x3e)

This register only exists if `sbaccess128` is 1.

If the bus master is busy then accesses set `sbbusyerror`, and don't do anything else.

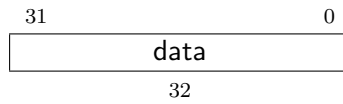


Field	Description	Access	Reset
data	Accesses bits 95:64 of sbddata (if the system bus is that wide).	R/W	0

3.12.26 System Bus Data 127:96 (sbddata3, at 0x3f)

This register only exists if [sbaccess128](#) is 1.

If the bus master is busy then accesses set [sbbusyerror](#), and don't do anything else.



Field	Description	Access	Reset
data	Accesses bits 127:96 of sbddata (if the system bus is that wide).	R/W	0

Chapter 4

Debug Mode inside the CPU

RISC-V Debug

Modifications to the RISC-V core to support debug are kept to a minimum. There is a special execution mode (Debug Mode) and a few extra CSRs. The DM takes care of the rest.

In order to be compliant with this specification an implementation must implement everything described in this section that is not explicitly listed as optional.

4.1 Debug Mode

Debug Mode is a special processor mode used only when a hart is halted for external debugging. How Debug Mode is implemented is not specified here.

When executing code from the optional Program Buffer, the hart stays in Debug Mode and the following apply:

1. All operations are executed at machine mode privilege level, except that MPRV in `mstatus` may be ignored according to `mprven`.
2. All interrupts (including NMI) are masked.
3. Exceptions don't update any registers. That includes `cause`, `epc`, `tval`, `dpc`, and `mstatus`. They do end execution of the Program Buffer.
4. No action is taken if a trigger matches.
5. Counters may be stopped, depending on `stopcount` in `dcscr`.
6. Timers may be stopped, depending on `stoptime` in `dcscr`.
7. The `wfi` instruction acts as a `nop`.
8. Almost all instructions that change the privilege level have undefined behavior. This includes `ecall`, `mret`, `sret`, and `uret`. (To change the privilege level, the debugger can write `prv` in `dcscr`). The only exception is `ebreak`. When that is executed in Debug Mode, it halts the hart again but without updating `dpc` or `dcscr`.
9. Completing Program Buffer execution is considered output for the purpose of `fence` instructions.
10. All control transfer instructions may act as illegal instructions if their destination is in the Program Buffer. If one such instruction acts as an illegal instruction, all such instructions

must act as an illegal instruction.

11. All control transfer instructions may act as illegal instructions if their destination is outside the Program Buffer. If one such instruction acts as an illegal instruction, all such instructions must act as an illegal instruction.
12. Instructions that depend on the value of the PC (e.g. `auipc`) may act as illegal instructions.
13. Effective XLEN is DXLEN.

In general, the debugger is expected to be able to simulate all the effects of MPRV. The exception is the case of Sv32 systems, which need MPRV functionality in order to access 34-bit physical addresses. Other systems are likely to tie `mprven` to 0.

4.2 Load-Reserved/Store-Conditional Instructions

The reservation registered by an `lr` instruction on a memory address may be lost when entering Debug Mode or while in Debug Mode. This means that there may be no forward progress if Debug Mode is entered between `lr` and `sc` pairs.

This is a behavior that debug users must be aware of. If they have a breakpoint set between a `lr` and `sc` pair, or are stepping through such code, the `sc` may never succeed. Fortunately in general use there will be very few instructions in such a sequence, and anybody debugging it will quickly notice that the reservation is not occurring. The solution in that case is to set a breakpoint on the first instruction after the `sc` and run to it. A higher level debugger may choose to automate this.

4.3 Wait for Interrupt Instruction

If halt is requested while `wfi` is executing, then the hart must leave the stalled state, completing this instruction's execution, and then enter Debug Mode.

4.4 Single Step

A debugger can cause a halted hart to execute a single instruction and then re-enter Debug Mode by setting `step` before setting `resumereq`.

If executing or fetching that instruction causes an exception, Debug Mode is re-entered immediately after the PC is changed to the exception handler and the appropriate `tval` and `cause` registers are updated.

If executing or fetching the instruction causes a trigger to fire, Debug Mode is re-entered immediately after that trigger has fired. In that case `cause` is set to 2 (trigger) instead of 4 (single step). Whether the instruction is executed or not depends on the specific configuration of the trigger.

If the instruction that is executed causes the PC to change to an address where an instruction fetch causes an exception, that exception does not occur until the next time the hart is resumed.

Similarly, a trigger at the new address does not fire until the hart actually attempts to execute that instruction.

If the instruction being stepped over is `wfi` and would normally stall the hart, then instead the instruction is treated as `nop`.

4.5 Reset

If the halt signal (driven by the hart's halt request bit in the Debug Module) or `resethaltreq` are asserted when a hart comes out of reset, the hart must enter Debug Mode before executing any instructions, but after performing any initialization that would usually happen before the first instruction is executed.

4.6 dret Instruction

To return from Debug Mode, a new instruction is defined: `dret`. It has an encoding of 0x7b200073. On harts which support this instruction, executing `dret` in Debug Mode changes `pc` to the value stored in `dpc`. The current privilege level is changed to that specified by `prv` in `dcsr`. The hart is no longer in debug mode.

Executing `dret` outside of Debug Mode causes an illegal instruction exception.

It is not necessary for the debugger to know whether an implementation supports `dret`, as the Debug Module will ensure that it is executed if necessary. It is defined in this specification only to reserve the opcode and allow for reusable Debug Module implementations.

4.7 XLEN

While in Debug Mode, XLEN is DXLEN. It is up to the debugger to determine the XLEN during normal program execution (by looking at `misa`) and to clearly communicate this to the user.

4.8 Core Debug Registers

The supported Core Debug Registers must be implemented for each hart that can be debugged. They are CSRs, accessible using the RISC-V `csr` opcodes and optionally also using abstract debug commands.

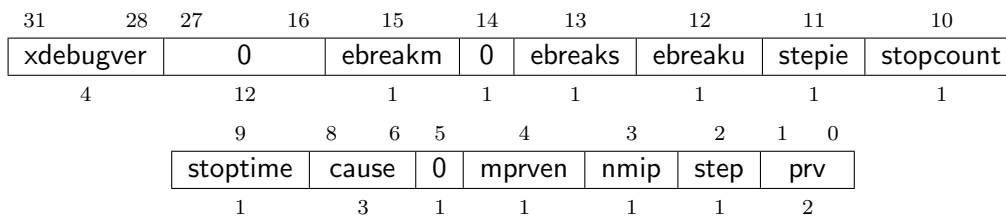
These registers are only accessible from Debug Mode.

Table 4.1: Core Debug Registers

Address	Name	Page
0x7b0	Debug Control and Status (dcsr)	42
0x7b1	Debug PC (dpc)	44
0x7b2	Debug Scratch Register 0 (dscratch0)	45
0x7b3	Debug Scratch Register 1 (dscratch1)	45

4.8.1 Debug Control and Status (**dcsr**, at 0x7b0)

cause priorities are assigned such that the least predictable events have the highest priority.



Field	Description	Access	Reset
xdebugver	0: There is no external debug support. 4: External debug support exists as it is described in this document. 15: There is external debug support, but it does not conform to any available version of this spec.	R	Preset
ebreakm	0: ebreak instructions in M-mode behave as described in the Privileged Spec. 1: ebreak instructions in M-mode enter Debug Mode.	R/W	0
ebreaks	0: ebreak instructions in S-mode behave as described in the Privileged Spec. 1: ebreak instructions in S-mode enter Debug Mode.	R/W	0
ebreaku	0: ebreak instructions in U-mode behave as described in the Privileged Spec. 1: ebreak instructions in U-mode enter Debug Mode.	R/W	0
stepie	0: Interrupts are disabled during single stepping. 1: Interrupts are enabled during single stepping. Implementations may hard wire this bit to 0. In that case interrupt behavior can be emulated by the debugger. The debugger must not change the value of this bit while the hart is running.	WARL	0

Continued on next page

Field	Description	Access	Reset
stopcount	0: Increment counters as usual. 1: Don't increment any counters while in Debug Mode or on ebreak instructions that cause entry into Debug Mode. These counters include the cycle and instret CSRs. This is preferred for most debugging scenarios. An implementation may hardwire this bit to 0 or 1.	WARL	Preset
stoptime	0: Increment timers as usual. 1: Don't increment any hart-local timers while in Debug Mode. An implementation may hardwire this bit to 0 or 1.	WARL	Preset
cause	Explains why Debug Mode was entered. When there are multiple reasons to enter Debug Mode in a single cycle, hardware should set cause to the cause with the highest priority. 1: An ebreak instruction was executed. (priority 3) 2: The Trigger Module caused a breakpoint exception. (priority 4, highest) 3: The debugger requested entry to Debug Mode using hltreq . (priority 1) 4: The hart single stepped because step was set. (priority 0, lowest) 5: The hart halted directly out of reset due to resethaltreq . It is also acceptable to report 3 when this happens. (priority 2) Other values are reserved for future use.	R	0
mprven	0: MPRV in mstatus is ignored in Debug Mode. 1: MPRV in mstatus takes effect in Debug Mode. Implementing this bit is optional. It may be tied to either 0 or 1.	WARL	Preset
nmip	When set, there is a Non-Maskable-Interrupt (NMI) pending for the hart. Since an NMI can indicate a hardware error condition, reliable debugging may no longer be possible once this bit becomes set. This is implementation-dependent.	R	0

Continued on next page

Field	Description	Access	Reset
step	When set and not in Debug Mode, the hart will only execute a single instruction and then enter Debug Mode. If the instruction does not complete due to an exception, the hart will immediately enter Debug Mode before executing the trap handler, with appropriate exception registers set. The debugger must not change the value of this bit while the hart is running.	R/W	0
prv	Contains the privilege level the hart was operating in when Debug Mode was entered. The encoding is described in Table 4.5. A debugger can change this value to change the hart's privilege level when exiting Debug Mode. Not all privilege levels are supported on all harts. If the encoding written is not supported or the debugger is not allowed to change to it, the hart may change to any supported privilege level.	R/W	3

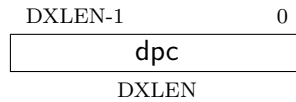
4.8.2 Debug PC (dpc, at 0x7b1)

Upon entry to debug mode, **dpc** is updated with the virtual address of the next instruction to be executed. The behavior is described in more detail in Table 4.3.

Table 4.3: Virtual address in DPC upon Debug Mode Entry

Cause	Virtual Address in DPC
ebreak	Address of the ebreak instruction
single step	Address of the instruction that would be executed next if no debugging was going on. Ie. pc + 4 for 32-bit instructions that don't change program flow, the destination PC on taken jumps/branches, etc.
trigger module	If timing is 0, the address of the instruction which caused the trigger to fire. If timing is 1, the address of the next instruction to be executed at the time that debug mode was entered.
halt request	Address of the next instruction to be executed at the time that debug mode was entered

When resuming, the hart's PC is updated to the virtual address stored in **dpc**. A debugger may write **dpc** to change where the hart resumes.



4.8.3 Debug Scratch Register 0 (dscratch0, at 0x7b2)

Optional scratch register that can be used by implementations that need it. A debugger must not write to this register unless [hartinfo](#) explicitly mentions it (the Debug Module may use this register internally).

4.8.4 Debug Scratch Register 1 (dscratch1, at 0x7b3)

Optional scratch register that can be used by implementations that need it. A debugger must not write to this register unless [hartinfo](#) explicitly mentions it (the Debug Module may use this register internally).

4.9 Virtual Debug Registers

A virtual register is one that doesn't exist directly in the hardware, but that the debugger exposes as if it does. Debug software should implement them, but hardware can skip this section. Virtual registers exist to give users access to functionality that's not part of standard debuggers without requiring them to carefully modify debug registers while the debugger is also accessing those same registers.

Table 4.4: Virtual Core Debug Registers

Address	Name	Page
virtual	Privilege Level (priv)	45

4.9.1 Privilege Level (priv, at virtual)

Users can read this register to inspect the privilege level that the hart was running in when the hart halted. Users can write this register to change the privilege level that the hart will run in when it resumes.

This register contains [prv](#) from [dcsr](#), but in a place that the user is expected to access. The user should not access [dcsr](#) directly, because doing so might interfere with the debugger.

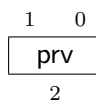


Table 4.5: Privilege Level Encoding

Encoding	Privilege Level
0	User/Application
1	Supervisor
3	Machine

Field	Description	Access	Reset
prv	Contains the privilege level the hart was operating in when Debug Mode was entered. The encoding is described in Table 4.5, and matches the privilege level encoding from the Privileged Spec. A user can write this value to change the hart's privilege level when exiting Debug Mode.	R/W	0

Chapter 5

Trigger Module

Triggers can cause a breakpoint exception, entry into Debug Mode, or a trace action without having to execute a special instruction. This makes them invaluable when debugging code from ROM. They can trigger on execution of instructions at a given memory address, or on the address/data in loads/stores. These are all features that can be useful without having the Debug Module present, so the Trigger Module is broken out as a separate piece that can be implemented separately.

A hart can be compliant with this specification without implementing any trigger functionality at all, but if it is implemented then it must conform to this section.

Triggers do not fire while in Debug Mode.

Each trigger may support a variety of features. A debugger can build a list of all triggers and their features as follows:

1. Write 0 to `tselect`.
2. Read back `tselect` and check that it contains the written value. If not, exit the loop.
3. Read `tinfo`.
4. If that caused an exception, the debugger must read `tdata1` to discover the type. (If `type` is 0, this trigger doesn't exist. Exit the loop.)
5. If `info` is 1, this trigger doesn't exist. Exit the loop.
6. Otherwise, the selected trigger supports the types discovered in `info`.
7. Repeat, incrementing the value in `tselect`.

The above algorithm reads back `tselect` so that implementations which have 2^n triggers only need to implement n bits of `tselect`.

The algorithm checks `tinfo` and `type` in case the implementation has m bits of `tselect` but fewer than 2^m triggers.

It is possible for a trigger with the “enter Debug Mode” action (1) and another trigger with the “raise a breakpoint exception” action (0) to fire at the same time. The preferred behavior is to have both actions take place. It is implementation-dependent which of the two happens first. This ensures both that the presence of an external debugger doesn't affect execution and that a trigger set by user code doesn't affect the external debugger. If this is not implemented, then the hart must enter Debug Mode and ignore the breakpoint exception. In the latter case, `hit` of the trigger

whose action is 0 must still be set, giving a debugger an opportunity to handle this case. What happens with trace actions when triggers with different actions are also firing is left to the trace specification.

5.1 Native M-Mode Triggers

Triggers can be used for native debugging. On a fully featured system triggers will be set using `u` or `s`, and when firing they can cause a breakpoint exception to trap to a more privileged mode. It is possible to set triggers natively to fire in M mode as well. In that case there is no higher privilege mode to trap to. When such a trigger causes a breakpoint exception while already in a trap handler, this will leave the system unable to resume normal execution.

On full-featured systems this is a remote corner case that can probably be ignored. On systems that only implement M mode, however, it is recommended to implement one of two solutions to this problem. This way triggers can be useful for native debugging of even M mode code.

The simple solution is to have the hardware prevent triggers with `action=0` from firing while in M mode and while `MIE` in `mstatus` is 0. Its limitation is that interrupts might be disabled at other times when a user might want triggers to fire.

A more complex solution is to implement `mte` and `mppte` in `tcontrol`. This solution has the benefit that it only disables triggers during the trap handler.

A user setting M mode triggers that cause breakpoint exceptions will have to be aware of any problems that might come up with the particular system they are working on.

5.2 Trigger Registers

These registers are CSRs, accessible using the RISC-V `csr` opcodes and optionally also using abstract debug commands.

Most trigger functionality is optional. All `tdata` registers follow write-any-read-legal semantics. If a debugger writes an unsupported configuration, the register will read back a value that is supported (which may simply be a disabled trigger). This means that a debugger must always read back values it writes to `tdata` registers, unless it already knows already what is supported. Writes to one `tdata` register may not modify the contents of other `tdata` registers, nor the configuration of any trigger besides the one that is currently selected.

The trigger registers are only accessible in machine and Debug Mode to prevent untrusted user code from causing entry into Debug Mode without the OS's permission.

In this section `XLEN` means `MXLEN` when in M-mode, and `DXLEN` when in Debug Mode. Note that this makes several of the fields in `tdata1` move around based on the current execution mode and value of `MXLEN`.

Table 5.1: [action](#) encoding

Value	Description
0	Raise a breakpoint exception. (Used when software wants to use the trigger module without an external debugger attached.)
1	Enter Debug Mode. (Only supported when the trigger's dmode is 1.)
2 – 5	Reserved for use by the trace specification.
other	Reserved for future use.

Table 5.2: Trigger Registers

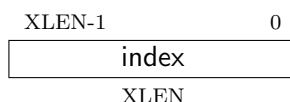
Address	Name	Page
0x7a0	Trigger Select (tselect)	49
0x7a1	Trigger Data 1 (tdata1)	50
0x7a1	Match Control (mcontrol)	53
0x7a1	Instruction Count (icount)	58
0x7a1	Interrupt Trigger (itrigger)	59
0x7a1	Exception Trigger (etrigger)	60
0x7a2	Trigger Data 2 (tdata2)	50
0x7a3	Trigger Data 3 (tdata3)	51
0x7a3	Trigger Extra (RV32) (textra32)	60
0x7a3	Trigger Extra (RV64) (textra64)	61
0x7a4	Trigger Info (tinfo)	51
0x7a5	Trigger Control (tcontrol)	51
0x7a8	Machine Context (mcontext)	52
0x7aa	Supervisor Context (scontext)	52

5.2.1 Trigger Select ([tselect](#), at 0x7a0)

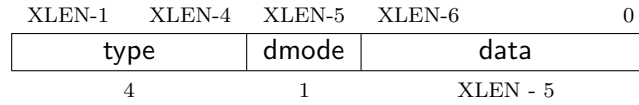
This register determines which trigger is accessible through the other trigger registers. The set of accessible triggers must start at 0, and be contiguous.

Writes of values greater than or equal to the number of supported triggers may result in a different value in this register than what was written. To verify that what they wrote is a valid index, debuggers can read back the value and check that [tselect](#) holds what they wrote.

Since triggers can be used both by Debug Mode and M-mode, the debugger must restore this register if it modifies it.



5.2.2 Trigger Data 1 (tdata1, at 0x7a1)

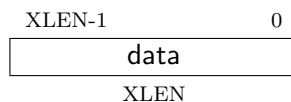


Field	Description	Access	Reset
type	0: There is no trigger at this tselect . 1: The trigger is a legacy SiFive address match trigger. These should not be implemented and aren't further documented here. 2: The trigger is an address/data match trigger. The remaining bits in this register act as described in mcontrol . 3: The trigger is an instruction count trigger. The remaining bits in this register act as described in icount . 4: The trigger is an interrupt trigger. The remaining bits in this register act as described in itrigger . 5: The trigger is an exception trigger. The remaining bits in this register act as described in etrigger . 15: This trigger exists (so enumeration shouldn't terminate), but is not currently available. Other values are reserved for future use.	R/W	Preset
dmode	0: Both Debug and M-mode can write the tdata registers at the selected tselect . 1: Only Debug Mode can write the tdata registers at the selected tselect . Writes from other modes are ignored. This bit is only writable from Debug Mode.	R/W	0
data	Trigger-specific data.	R/W	Preset

5.2.3 Trigger Data 2 (tdata2, at 0x7a2)

Trigger-specific data.

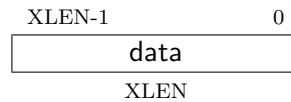
If XLEN is less than DXLEN, writes to this register are sign-extended.



5.2.4 Trigger Data 3 (tdata3, at 0x7a3)

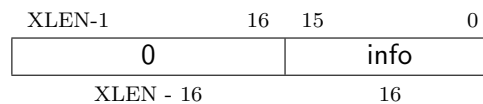
Trigger-specific data.

If XLEN is less than DXLEN, writes to this register are sign-extended.



5.2.5 Trigger Info (tinfo, at 0x7a4)

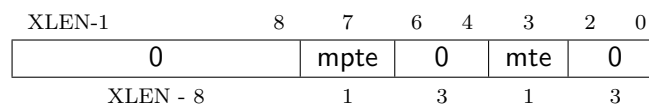
This entire register is read-only.



Field	Description	Access	Reset
info	One bit for each possible type enumerated in tdata1. Bit N corresponds to type N. If the bit is set, then that type is supported by the currently selected trigger. If the currently selected trigger doesn't exist, this field contains 1. If type is not writable, this register may be unimplemented, in which case reading it causes an illegal instruction exception. In this case the debugger can read the only supported type from tdata1.	R	Preset

5.2.6 Trigger Control (tcontrol, at 0x7a5)

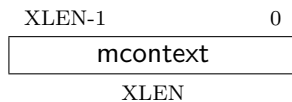
This optional register is one solution to a problem regarding triggers with action=0 firing in M-mode trap handlers. See [Section 5.1](#) for more details.



Field	Description	Access	Reset
mp te	M-mode previous trigger enable field. When a trap into M-mode is taken, mp te is set to the value of mt e.	R/W	0
mt e	M-mode trigger enable field. 0: Triggers with action=0 do not match/fire while the hart is in M-mode. 1: Triggers do match/fire while the hart is in M-mode. When a trap into M-mode is taken, mt e is set to 0. When mret is executed, mt e is set to the value of mp te.	R/W	0

5.2.7 Machine Context (**mcontext**, at 0x7a8)

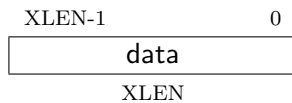
This register is only writable in M mode and Debug Mode.



Field	Description	Access	Reset
mcontext	Machine mode software can write a context number to this register, which can be used to set triggers that only fire in that specific context. An implementation may tie any number of upper bits in this field to 0. It's recommended to implement no more than 6 bits on RV32, and 13 on RV64.	R/W	0

5.2.8 Supervisor Context (**scontext**, at 0x7aa)

This register is only writable in S mode, M mode and Debug Mode.



Field	Description	Access	Reset
maskmax	Specifies the largest naturally aligned powers-of-two (NAPOT) range supported by the hardware when match is 1. The value is the logarithm base 2 of the number of bytes in that range. A value of 0 indicates that only exact value matches are supported (one byte range). A value of 63 corresponds to the maximum NAPOT range, which is 2^{63} bytes in size.	R	Preset
sizehi	This field only exists if XLEN is greater than 32. In that case it extends size . If it does not exist then hardware operates as if the field contains 0.	R/W	0
hit	If this optional bit is implemented, the hardware sets it when this trigger matches. The trigger's user can set or clear it at any time. It is used to determine which trigger(s) matched. If the bit is not implemented, it is always 0 and writing it has no effect.	R/W	0
select	0: Perform a match on the virtual address. 1: Perform a match on the data value loaded or stored, or the instruction executed.	R/W	0

Continued on next page

Field	Description	Access	Reset
timing	0: The action for this trigger will be taken just before the instruction that triggered it is executed, but after all preceding instructions are committed. 1: The action for this trigger will be taken after the instruction that triggered it is executed. It should be taken before the next instruction is executed, but it is better to implement triggers and not implement that suggestion than to not implement them at all. Most hardware will only implement one timing or the other, possibly dependent on select, execute, load, and store. This bit primarily exists for the hardware to communicate to the debugger what will happen. Hardware may implement the bit fully writable, in which case the debugger has a little more control. Data load triggers with timing of 0 will result in the same load happening again when the debugger lets the hart run. For data load triggers, debuggers must first attempt to set the breakpoint with timing of 1. A chain of triggers that don't all have the same timing value will never fire (unless consecutive instructions match the appropriate triggers). If a trigger with timing of 0 matches, it is implementation-dependent whether that prevents a trigger with timing of 1 matching as well.	R/W	0

Continued on next page

Field	Description	Access	Reset
size _{lo}	This field contains the 2 low bits of size. The high bits come from size_{hi}. The combined value is interpreted as follows: 0: The trigger will attempt to match against an access of any size. The behavior is only well-defined if <code>select = 0</code>, or if the access size is XLEN. 1: The trigger will only match against 8-bit memory accesses. 2: The trigger will only match against 16-bit memory accesses or execution of 16-bit instructions. 3: The trigger will only match against 32-bit memory accesses or execution of 32-bit instructions. 4: The trigger will only match against execution of 48-bit instructions. 5: The trigger will only match against 64-bit memory accesses or execution of 64-bit instructions. 6: The trigger will only match against execution of 80-bit instructions. 7: The trigger will only match against execution of 96-bit instructions. 8: The trigger will only match against execution of 112-bit instructions. 9: The trigger will only match against 128-bit memory accesses or execution of 128-bit instructions.	R/W	0
action	The action to take when the trigger fires. The values are explained in Table 5.1 .	R/W	0

Continued on next page

Field	Description	Access	Reset
chain	0: When this trigger matches, the configured action is taken. 1: While this trigger does not match, it prevents the trigger with the next index from matching. A trigger chain starts on the first trigger with chain = 1 after a trigger with chain = 0, or simply on the first trigger if that has chain = 1. It ends on the first trigger after that which has chain = 0. This final trigger is part of the chain. The action on all but the final trigger is ignored. The action on that final trigger will be taken if and only if all the triggers in the chain match at the same time. Because chain affects the next trigger, hardware must zero it in writes to mcontrol that set dmode to 0 if the next trigger has dmode of 1. In addition hardware should ignore writes to mcontrol that set dmode to 1 if the previous trigger has both dmode of 0 and chain of 1. Debuggers must avoid the latter case by checking chain on the previous trigger if they're writing mcontrol. Implementations that wish to limit the maximum length of a trigger chain (eg. to meet timing requirements) may do so by zeroing chain in writes to mcontrol that would make the chain too long.	R/W	0
match	0: Matches when the value equals tdata2. 1: Matches when the top M bits of the value match the top M bits of tdata2. M is XLEN-1 minus the index of the least-significant bit containing 0 in tdata2. 2: Matches when the value is greater than (unsigned) or equal to tdata2. 3: Matches when the value is less than (unsigned) tdata2. 4: Matches when the lower half of the value equals the lower half of tdata2 after the lower half of the value is ANDed with the upper half of tdata2. 5: Matches when the upper half of the value equals the lower half of tdata2 after the upper half of the value is ANDed with the upper half of tdata2. Other values are reserved for future use.	R/W	0
m	When set, enable this trigger in M-mode.	R/W	0
s	When set, enable this trigger in S-mode.	R/W	0
u	When set, enable this trigger in U-mode.	R/W	0

Continued on next page

Field	Description	Access	Reset
execute	When set, the trigger fires on the virtual address or opcode of an instruction that is executed.	R/W	0
store	When set, the trigger fires on the virtual address or data of a store.	R/W	0
load	When set, the trigger fires on the virtual address or data of a load.	R/W	0

5.2.10 Instruction Count (icount, at 0x7a1)

This register is accessible as `tdata1` when `type` is 3.

This trigger type is intended to be used as a single step that's useful both for external debuggers and for software monitor programs. For that case it is not necessary to support `count` greater than 1. The only two combinations of the mode bits that are useful in those scenarios are `u` by itself, or `m`, `s`, and `u` all set.

If the hardware limits `count` to 1, and changes mode bits instead of decrementing `count`, this register can be implemented with just 2 bits. One for `u`, and one for `m` and `s` tied together. If only the external debugger or only a software monitor needs to be supported, a single bit is enough.

XLEN-1	XLEN-4	XLEN-5	XLEN-6	25	24	23	10	9	8	7	6	5	0
type	dmode	0	hit	count	m	0	s	u	action				
4	1	XLEN - 30	1	14	1	1	1	1	6				

Field	Description	Access	Reset
hit	If this optional bit is implemented, the hardware sets it when this trigger matches. The trigger's user can set or clear it at any time. It is used to determine which trigger(s) matched. If the bit is not implemented, it is always 0 and writing it has no effect.	R/W	0
count	When count is decremented to 0, the trigger fires. Instead of changing <code>count</code> from 1 to 0, it is also acceptable for hardware to clear <code>m</code> , <code>s</code> , and <code>u</code> . This allows <code>count</code> to be hard-wired to 1 if this register just exists for single step.	R/W	1
m	When set, every instruction completed or exception taken in M-mode decrements <code>count</code> by 1.	R/W	0
s	When set, every instruction completed or exception taken in S-mode decrements <code>count</code> by 1.	R/W	0
u	When set, every instruction completed or exception taken in U-mode decrements <code>count</code> by 1.	R/W	0

Continued on next page

Field	Description	Access	Reset
action	The action to take when the trigger fires. The values are explained in Table 5.1.	R/W	0

5.2.11 Interrupt Trigger (ittrigger, at 0x7a1)

This register is accessible as `tdata1` when `type` is 4.

This trigger may fire on any of the interrupts configurable in `mie` (described in the Privileged Spec). The interrupts to fire on are configured by setting the same bit in `tdata2` as would be set in `mie` to enable the interrupt.

Hardware may only support a subset of interrupts for this trigger. A debugger must read back `tdata2` after writing it to confirm the requested functionality is actually supported.

The trigger only fires if the hart takes a trap because of the interrupt. (E.g. it does not fire when a timer interrupt occurs but that interrupt is not enabled in `mie`.)

When the trigger fires, all CSRs are updated as defined by the Privileged Spec, and the requested action is taken just before the first instruction of the interrupt/exception handler is executed.

XLEN-1	XLEN-4	XLEN-5	XLEN-6	XLEN-7	10	9	8	7	6	5	0
type	dmode	hit	0	m	0	s	u	action			
4	1	1	XLEN - 16	1	1	1	1	6			

Field	Description	Access	Reset
hit	If this optional bit is implemented, the hardware sets it when this trigger matches. The trigger's user can set or clear it at any time. It is used to determine which trigger(s) matched. If the bit is not implemented, it is always 0 and writing it has no effect.	R/W	0
m	When set, enable this trigger for interrupts that are taken from M mode.	R/W	0
s	When set, enable this trigger for interrupts that are taken from S mode.	R/W	0
u	When set, enable this trigger for interrupts that are taken from U mode.	R/W	0
action	The action to take when the trigger fires. The values are explained in Table 5.1.	R/W	0

5.2.12 Exception Trigger (etrigger, at 0x7a1)

This register is accessible as `tdata1` when `type` is 5.

This trigger may fire on up to XLEN of the Exception Codes defined in `mcause` (described in the Privileged Spec, with Interrupt=0). Those causes are configured by writing the corresponding bit in `tdata2`. (E.g. to trap on an illegal instruction, the debugger sets bit 2 in `tdata2`.)

Hardware may support only a subset of exceptions. A debugger must read back `tdata2` after writing it to confirm the requested functionality is actually supported.

When the trigger fires, all CSRs are updated as defined by the Privileged Spec, and the requested action is taken just before the first instruction of the interrupt/exception handler is executed.

XLEN-1	XLEN-4	XLEN-5	XLEN-6	XLEN-7	10	9	8	7	6	5	0
type	dmode	hit	0			m	0	s	u	action	
4	1	1	XLEN - 16			1	1	1	1	6	

Field	Description	Access	Reset
hit	If this optional bit is implemented, the hardware sets it when this trigger matches. The trigger's user can set or clear it at any time. It is used to determine which trigger(s) matched. If the bit is not implemented, it is always 0 and writing it has no effect.	R/W	0
m	When set, enable this trigger for exceptions that are taken from M mode.	R/W	0
s	When set, enable this trigger for exceptions that are taken from S mode.	R/W	0
u	When set, enable this trigger for exceptions that are taken from U mode.	R/W	0
action	The action to take when the trigger fires. The values are explained in Table 5.1.	R/W	0

5.2.13 Trigger Extra (RV32) (textra32, at 0x7a3)

This register is accessible as `tdata3` when `type` is 2, 3, 4, or 5.

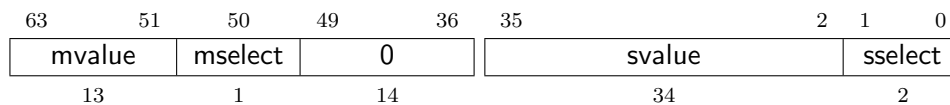
All functionality in this register is optional. The `value` bits may tie any number of upper bits to 0. The `select` bits may only support 0 (ignore).

31	26	25	24	18	17	2	1	0
mvalue	mselect	0			svalue	sselect		
6	1	7			16	2		

Field	Description	Access	Reset
mvalue	Data used together with mselect .	R/W	0
mselect	0: Ignore mvalue . 1: This trigger will only match if the low bits of mcontext equal mvalue .	WARL	0
svalue	Data used together with sselect .	R/W	0
sselect	0: Ignore svalue . 1: This trigger will only match if the low bits of scontext equal svalue . 2: This trigger will only match if ASID in satp equals the lower ASIDMAX (defined in the Privileged Spec) bits of svalue .	WARL	0

5.2.14 Trigger Extra (RV64) (textra64, at 0x7a3)

This is the layout of `textra` if XLEN is 64. The fields are defined above, in [textra32](#).



Chapter 6

Debug Transport Module (DTM)

Debug Transport Modules provide access to the DM over one or more transports (e.g. JTAG or USB).

There may be multiple DTMs in a single platform. Ideally every component that communicates with the outside world includes a DTM, allowing a platform to be debugged through every transport it supports. For instance a USB component could include a DTM. This would trivially allow any platform to be debugged over USB. All that is required is that the USB module already in use also has access to the Debug Module Interface.

Using multiple DTMs at the same time is not supported. It is left to the user to ensure this does not happen.

This specification defines a JTAG DTM in Section 6.1. Additional DTMs may be added in future versions of this specification.

An implementation can be compliant with this specification without implementing any of this section. In that case it must be advertised as conforming to “RISC-V Debug Specification 0.13.2, with custom DTM.” If the JTAG DTM described here is implemented, it must be advertised as conforming to the “RISC-V Debug Specification 0.13.2, with JTAG DTM.”

6.1 JTAG Debug Transport Module

This Debug Transport Module is based around a normal JTAG Test Access Port (TAP). The JTAG TAP allows access to arbitrary JTAG registers by first selecting one using the JTAG instruction register (IR), and then accessing it through the JTAG data register (DR).

6.1.1 JTAG Background

JTAG refers to IEEE Std 1149.1-2013. It is a standard that defines test logic that can be included in an integrated circuit to test the interconnections between integrated circuits, test the integrated

circuit itself, and observe or modify circuit activity during the components normal operation. This specification uses the latter functionality. The JTAG standard defines a Test Access Port (TAP) that can be used to read and write a few custom registers, which can be used to communicate with debug hardware in a component.

6.1.2 JTAG DTM Registers

JTAG TAPs used as a DTM must have an **IR of at least 5 bits**. When the TAP is reset, IR must default to **00001**, selecting the IDCODE instruction. A full list of JTAG registers along with their encoding is in Table 6.1. If the IR actually has more than 5 bits, then the encodings in Table 6.1 should be extended with 0's in their most significant bits. The only regular JTAG registers a debugger might use are BYPASS and IDCODE, but this specification leaves IR space for many other standard JTAG instructions. Unimplemented instructions must select the BYPASS register.

Table 6.1: JTAG DTM TAP Registers

Address	Name	Description	Page
0x00	BYPASS	JTAG recommends this encoding	64
0x01	IDCODE	JTAG recommends this encoding	
0x10	DTM Control and Status (dtmcs)	For Debugging	
0x11	Debug Module Interface Access (dmi)	For Debugging	
0x12	Reserved (BYPASS)	Reserved for future RISC-V debugging	
0x13	Reserved (BYPASS)	Reserved for future RISC-V debugging	
0x14	Reserved (BYPASS)	Reserved for future RISC-V debugging	
0x15	Reserved (BYPASS)	Reserved for future RISC-V standards	
0x16	Reserved (BYPASS)	Reserved for future RISC-V standards	
0x17	Reserved (BYPASS)	Reserved for future RISC-V standards	
0x1f	BYPASS	JTAG requires this encoding	65

6.1.3 IDCODE (at 0x01)

This register is selected (in IR) when the TAP state machine is reset. Its definition is exactly as defined in IEEE Std 1149.1-2013.

This entire register is read-only.

31	28	27	12	11	1	0
Version		PartNumber		Manufld		1
4		16		11		1

Field	Description	Access	Reset
Version	Identifies the release version of this part.	R	Preset

Continued on next page

Field	Description	Access	Reset
PartNumber	Identifies the designer's part number of this part.	R	Preset
Manufld	Identifies the designer/manufacturer of this part. Bits 6:0 must be bits 6:0 of the designer/manufacturer's Identification Code as assigned by JEDEC Standard JEP106. Bits 10:7 contain the modulo-16 count of the number of continuation characters (0x7f) in that same Identification Code.	R	Preset

6.1.4 DTM Control and Status (dtmcs, at 0x10)

The size of this register will remain constant in future versions so that a debugger can always determine the version of the DTM.

31	18	17	16	15	14	12	11	10	9	4	3	0
0	dmihardreset	dmireset	0	idle	dmistat	abits	version					
14	1	1	1	3	2	6	4					

Field	Description	Access	Reset
dmihardreset	Writing 1 to this bit does a hard reset of the DTM, causing the DTM to forget about any outstanding DMI transactions. In general this should only be used when the Debugger has reason to expect that the outstanding DMI transaction will never complete (e.g. a reset condition caused an inflight DMI transaction to be cancelled).	W1	-
dmireset	Writing 1 to this bit clears the sticky error state and allows the DTM to retry or complete the previous transaction.	W1	-
idle	This is a hint to the debugger of the minimum number of cycles a debugger should spend in Run-Test/Idle after every DMI scan to avoid a 'busy' return code (dmistat of 3). A debugger must still check dmistat when necessary. 0: It is not necessary to enter Run-Test/Idle at all. 1: Enter Run-Test/Idle and leave it immediately. 2: Enter Run-Test/Idle and stay there for 1 cycle before leaving. And so on.	R	Preset

Continued on next page

Field	Description	Access	Reset
dmistat	0: No error. 1: Reserved. Interpret the same as 2. 2: An operation failed (resulted in op of 2). 3: An operation was attempted while a DMI access was still in progress (resulted in op of 3).	R	0
abits	The size of address in dmi .	R	Preset
version	0: Version described in spec version 0.11. 1: Version described in spec version 0.13. 15: Version not described in any available version of this spec.	R	1

6.1.5 Debug Module Interface Access (dmi, at 0x11)

This register allows access to the **Debug Module Interface (DMI)**.

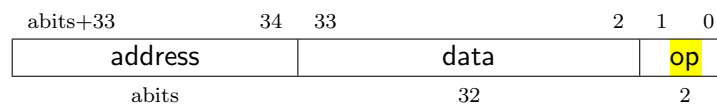
In **Update-DR**, the DTM starts the operation specified in **op** unless the current status reported in **op** is sticky.

In **Capture-DR**, the DTM updates **data** with the result from that operation, updating **op** if the current **op** isn't sticky.

See Section B.1 and Table ?? for examples of how this is used.

The still-in-progress status is sticky to accommodate debuggers that batch together a number of scans, which must all be executed or stop as soon as there's a problem.

For instance a series of scans may write a Debug Program and execute it. If one of the writes fails but the execution continues, then the Debug Program may hang or have other unexpected side effects.



Field	Description	Access	Reset
address	Address used for DMI access. In Update-DR this value is used to access the DM over the DMI.	R/W	0
data	The data to send to the DM over the DMI during Update-DR, and the data returned from the DM as a result of the previous operation.	R/W	0

Continued on next page

Field	Description	Access	Reset
op	When the debugger writes this field, it has the following meaning: 0: Ignore data and address. (nop) Don't send anything over the DMI during Update-DR. This operation should never result in a busy or error response. The address and data reported in the following Capture-DR are undefined. 1: Read from address. (read) 2: Write data to address. (write) 3: Reserved. When the debugger reads this field, it means the following: 0: The previous operation completed successfully. 1: Reserved. 2: A previous operation failed. The data scanned into dmi in this access will be ignored. This status is sticky and can be cleared by writing dmireset in dtmcs. This indicates that the DM itself responded with an error. There are no specified cases in which the DM would respond with an error, and DMI is not required to support returning errors. 3: An operation was attempted while a DMI request is still in progress. The data scanned into dmi in this access will be ignored. This status is sticky and can be cleared by writing dmireset in dtmcs. If a debugger sees this status, it needs to give the target more TCK edges between Update-DR and Capture-DR. The simplest way to do that is to add extra transitions in Run-Test/Idle.	R/W	0

6.1.6 BYPASS (at 0x1f)

1-bit register that has no effect. It is used when a debugger does not want to communicate with this TAP.

This entire register is read-only.



6.1.7 Recommended JTAG Connector

To make it easy to acquire debug hardware, this spec recommends a connector that is compatible with the MIPI-10 .05 inch connector specification, as described in the MIPI Alliance Recommendation for Debug and Trace Connectors, Version 1.10.00, 16 March 2011.

The connector has .05 inch spacing, gold-plated male header with .016 inch thick hardened copper or beryllium bronze square posts (SAMTEC FTSH or equivalent). Female connectors are compatible 20 μ m gold connectors.

Viewing the male header from above (the pins pointing at your eye), a target's connector looks as it does in Table 6.5. The function of each pin is described in Table 6.7.

Table 6.5: MIPI-10 Connector Diagram

VREF DEBUG	1	2	TMS
GND	3	4	TCK
GND	5	6	TDO
GND or KEY	7	8	TDI
GND	9	10	nRESET

If a platform requires nTRST then it is permissible to reuse the nRESET pin as the nTRST signal. If a platform requires both system reset and TAP reset, the MIPI-20 connector should be used. Its physical connector is virtually identical to MIPI-10, except that it's twice as long, supporting twice as many pins. Its connector is show in Table 6.6.

Table 6.6: MIPI-20 Connector Diagram

VREF DEBUG	1	2	TMS
GND	3	4	TCK
GND	5	6	TDO
GND or KEY	7	8	TDI
GND	9	10	nRESET
GND	11	12	RTCK
GND	13	14	nTRST_PD
GND	15	16	nTRST
GND	17	18	DBGREQ
GND	19	20	DBGACK

The same connectors can be used for 2-wire cJTAG. In that case TMS is used for TMS0, and TCK is used for TCK0.

Table 6.7: JTAG Connector Pinout

1	VREF DEBUG	Reference voltage for logic high.
2	TMS	JTAG TMS signal, driven by the debug adapter.
4	TCK	JTAG TCK signal, driven by the debug adapter.
6	TDO	JTAG TDO signal, driven by the target.
7	GND or KEY	This pin may be cut on the male and plugged on the female header to ensure the header is always plugged in correctly. It is, however, recommended to use this pin as an additional ground, to allow for fastest TCK speeds. A shrouded connector should be used to prevent the cable from being plugged in incorrectly.
8	TDI	JTAG TDI signal, driven by the debug adapter.
10	nRESET	Active-low reset signal, driven by the debug adapter. Asserting reset should reset any RISC-V cores as well as any other peripherals on the PCB. It should not reset the debug logic. This pin is optional but strongly encouraged. If necessary, this pin could be used as nTRST instead. nRESET should never be connected to the TAP reset, otherwise the debugger might not be able to debug through a reset to discover the cause of a crash or to maintain execution control after the reset.
12	RTCK	Return test clock, driven by the target. A target may relay the TCK signal here once it has processed it, allowing a debugger to adjust its TCK frequency in response.
14	nTRST.PD	Test reset pull-down (optional), driven by the debug adapter. Same function as nTRST, but with pull-down resistor on target.
16	nTRST	Test reset (optional), driven by the debug adapter. Used to reset the JTAG TAP Controller.
18	TRIGIN	Not used, driven low by the debug adapter.
20	TRIGOUT	Not used, driven by the target.

Appendix A

Hardware Implementations

Below are two possible implementations. A designer could choose one, mix and match, or come up with their own design.

A.1 Abstract Command Based

Halting happens by stalling the hart execution pipeline.

Muxes on the register file(s) allow for accessing GPRs and CSRs using the Access Register abstract command.

Memory is accessed using the Abstract Access Memory command or through System Bus Access.

This implementation could allow a debugger to collect information from the hart even when that hart is unable to execute instructions.

A.2 Execution Based

This implementation only implements the Access Register abstract command for GPRs on a halted hart, and relies on the Program Buffer for all other operations. It uses the hart's existing pipeline and ability to execute from arbitrary memory locations to avoid modifications to a hart's datapath.

When the halt request bit is set, the Debug Module raises a special interrupt to the selected harts. This interrupt causes each hart to enter Debug Mode and jump to a defined memory region that is serviced by the DM. When taking this exception, `pc` is saved to `dpc` and `cause` is updated in `dcsr`.

The code in the Debug Module causes the hart to execute a “park loop.” In the park loop the hart writes its `mhartid` to a memory location within the Debug Module to indicate that it is halted. To allow the DM to individually control one out of several halted harts, each hart polls for flags in a DM-controlled memory location to determine whether the debugger wants it to execute the Program Buffer or perform a resume.

To execute an abstract command, the DM first populates some internal words of program buffer according to `command`. When `transfer` is set, the DM populates these words with `lw <gpr>, 0x400(zero)` or `sw 0x400(zero), <gpr>`. 64- and 128-bit accesses use `ld/sd` and `lq/sq` respectively. If `transfer` is not set, the DM populates these instructions as `nops`. If `execute` is set, execution continues to the debugger-controlled Program Buffer, otherwise the DM causes a `ebreak` to execute immediately.

When `ebreak` is executed (indicating the end of the Program Buffer code) the hart returns to its park loop. If an exception is encountered, the hart jumps to a debug exception address within the Debug Module. The code at that address causes the hart to write to an address in the Debug Module which indicates exception. This address is considered I/O for `fence` instructions (see #9 on page 39). Then the hart jumps back to the park loop. The DM infers from the write that there was an exception, and sets `cmderr` appropriately.

To resume execution, the debug module sets a flag which causes the hart to execute a `dret`. When `dret` is executed, `pc` is restored from `dpc` and normal execution resumes at the privilege set by `prv`.

`data0` etc. are mapped into regular memory at an address relative to `zero` with only a 12-bit `imm`. The exact address is an implementation detail that a debugger must not rely on. For example, the `data` registers might be mapped to `0x400`.

For additional flexibility, `progbuf0`, etc. are mapped into regular memory immediately preceding `data0`, in order to form a contiguous region of memory which can be used for either program execution or data transfer.

Appendix B

Debugger Implementation

This section details how an external debugger might use the described debug interface to perform some common operations on RISC-V cores using the JTAG DTM described in Section 6.1. All these examples assume a 32-bit core but it should be easy to adapt the examples to 64- or 128-bit cores.

To keep the examples readable, they all assume that everything succeeds, and that they complete faster than the debugger can perform the next access. This will be the case in a typical JTAG setup. However, the debugger must always check the sticky error status bits after performing a sequence of actions. If it sees any that are set, then it should attempt the same actions again, possibly while adding in some delay, or explicit checks for status bits.

B.1 Debug Module Interface Access

To read an arbitrary Debug Module register, select `dmi`, and scan in a value with `op` set to 1, and `address` set to the desired register address. In Update-DR the operation will start, and in Capture-DR its results will be captured into `data`. If the operation didn't complete in time, `op` will be 3 and the value in `data` must be ignored. The busy condition must be cleared by writing `dmireset` in `dtmcs`, and then the second scan must be performed again. This process must be repeated until `op` returns 0. In later operations the debugger should allow for more time between Capture-DR and Update-DR.

To write an arbitrary Debug Bus register, select `dmi`, and scan in a value with `op` set to 2, and `address` and `data` set to the desired register address and data respectively. From then on everything happens exactly as with a read, except that a write is performed instead of the read.

It should almost never be necessary to scan IR, avoiding a big part of the inefficiency in typical JTAG use.

B.2 Checking for Halted Harts

A user will want to know as quickly as possible when a hart is halted (e.g. due to a breakpoint). To efficiently determine which harts are halted when there are many harts, the debugger uses the `haltsum` registers. Assuming the maximum number of harts exist, first it checks `haltsum3`. For each bit set there, it writes `hartsel`, and checks `haltsum2`. This process repeats through `haltsum1` and `haltsum0`. Depending on how many harts exist, the process should start at one of the lower `haltsum` registers.

B.3 Halting

To halt one or more harts, the debugger selects them, sets `haltreq`, and then waits for `allhalted` to indicate the harts are halted. Then it can clear `haltreq` to 0, or leave it high to catch a hart that resets while halted.

B.4 Running

First, the debugger should restore any registers that it has overwritten. Then it can let the selected harts run by setting `resumereq`. Once `allresumeack` is set, the debugger knows the hart has resumed, and it can clear `resumereq`. Harts might halt very quickly after resuming (e.g. by hitting a software breakpoint) so the debugger cannot use `allhalted`/`anyhalted` to check whether the hart resumed.

B.5 Single Step

Using the hardware single step feature is almost the same as regular running. The debugger just sets `step` in `dcsr` before letting the hart run. The hart behaves exactly as in the running case, except that interrupts may be disabled (depending on `stepie`) and it only fetches and executes a single instruction before re-entering Debug Mode.

B.6 Accessing Registers

B.6.1 Using Abstract Command

Read `s0` using abstract command:

Op	Address	Value	Comment
Write	<code>command</code>	<code>aarsize = 2, transfer, regno = 0x1008</code>	Read <code>s0</code>
Read	<code>data0</code>	-	Returns value that was in <code>s0</code>

Write `mstatus` using abstract command:

Op	Address	Value	Comment
Write	<code>data0</code>	new value	
Write	<code>command</code>	<code>aarsize = 2, transfer, write, regno = 0x300</code>	Write <code>mstatus</code>

B.6.2 Using Program Buffer

Abstract commands are used to exchange data with GPRs. Using this mechanism, other registers can be accessed by moving their value into/out of GPRs.

Write `mstatus` using program buffer:

Op	Address	Value	Comment
Write	<code>progbuf0</code>	<code>csrw s0, MSTATUS</code>	
Write	<code>progbuf1</code>	<code>ebreak</code>	
Write	<code>data0</code>	new value	
Write	<code>command</code>	<code>aarsize = 2, postexec, transfer, write, regno = 0x1008</code>	Write <code>s0</code> , then execute program buffer

Read `f1` using program buffer:

Op	Address	Value	Comment
Write	<code>progbuf0</code>	<code>fmv.x.s s0, f1</code>	
Write	<code>progbuf1</code>	<code>ebreak</code>	
Write	<code>command</code>	<code>postexec</code>	Execute program buffer
Write	<code>command</code>	<code>transfer, regno = 0x1008</code>	read <code>s0</code>
Read	<code>data0</code>	-	Returns the value that was in <code>f1</code>

B.7 Reading Memory

B.7.1 Using System Bus Access

With system bus access, addresses are physical system bus addresses.

Read a word from memory using system bus access:

Op	Address	Value	Comment
Write	<code>sbc</code>	<code>sbaccess = 2, sbreadonaddr</code>	Setup
Write	<code>sbaddress0</code>	address	
Read	<code>sbdatab0</code>	-	Value read from memory

Read block of memory using system bus access:

Op	Address	Value	Comment
Write	<code>sbc</code>	<code>sbaccess = 2, sbreadonaddr, sbreadondata, sbautoincrement</code>	Turn on autoread and autoincrement
Write	<code>sbaddress0</code>	address	Writing address triggers read and increment
Read	<code>sbddata0</code>	-	Value read from memory
Read	<code>sbddata0</code>	-	Next value read from memory
...	...	...	...
Write	<code>sbc</code>	0	Disable autoread
Read	<code>sbddata0</code>	-	Get last value read from memory.

B.7.2 Using Program Buffer

Through the Program Buffer, the hart performs the memory accesses. Addresses are physical or virtual (depending on `mprven` and other system configuration).

Read a word from memory using program buffer:

Op	Address	Value	Comment
Write	<code>progbuf0</code>	<code>lw s0, 0(s0)</code>	
Write	<code>progbuf1</code>	<code>ebreak</code>	
Write	<code>data0</code>	address	
Write	<code>command</code>	<code>write, postexec, regno = 0x1008</code>	Write <code>s0</code> , then execute program buffer
Write	<code>command</code>	<code>regno = 0x1008</code>	Read <code>s0</code>
Read	<code>data0</code>	-	Value read from memory

Read block of memory using program buffer:

Op	Address	Value	Comment
Write	progbuf0	lw s1, 0(s0)	
Write	progbuf1	addi s0, s0, 4	
Write	progbuf2	ebreak	
Write	data0	address	
Write	command	write, postexec , regno = 0x1008	Write s0, then execute program buffer
Write	command	postexec , regno = 0x1009	Read s1, then execute program buffer
Write	abstractauto	autoexecdata [0]	Set autoexecdata [0]
Read	data0	-	Get value read from memory, then execute program buffer
Read	data0	-	Get next value read from memory, then execute program buffer
...	...	...	...
Write	abstractauto	0	Clear autoexecdata [0]
Read	data0	-	Get last value read from memory.

B.7.3 Using Abstract Memory Access

Abstract memory accesses act as if they are performed by the hart, although the actual implementation may differ.

Read a word from memory using abstract memory access:

Op	Address	Value	Comment
Write	data1	address	
Write	command	cmdtype=2, aamsize =2	
Read	data0	-	Value read from memory

Read block of memory using abstract memory access:

Op	Address	Value	Comment
Write	abstractauto	1	Re-execute the command when data0 is accessed
Write	data1	address	
Write	command	cmdtype=2, aamsize =2, aampostincrement =1	
Read	data0	-	Read value, and trigger reading of next address
...	...	...	...
Write	abstractauto	0	Disable auto-exec
Read	data0	-	Get last value read from memory.

B.8 Writing Memory

B.8.1 Using System Bus Access

With system bus access, addresses are physical system bus addresses.

Write a word to memory using system bus access:

Op	Address	Value	Comment
Write	sbaddress0	address	
Write	sbdata0	value	

Write a block of memory using system bus access:

Op	Address	Value	Comment
Write	sbcs	sbaccess = 2, sbautoincrement	Turn on autoincrement
Write	sbaddress0	address	
Write	sbdata0	value0	
Write	sbdata0	value1	
...	...	...	...
Write	sbdata0	valueN	

B.8.2 Using Program Buffer

Through the Program Buffer, the hart performs the memory accesses. Addresses are physical or virtual (depending on [mprven](#) and other system configuration).

Write a word to memory using program buffer:

Op	Address	Value	Comment
Write	progbuf0	<code>sw s1, 0(s0)</code>	
Write	progbuf1	<code>ebreak</code>	
Write	data0	address	
Write	command	<code>write, regno = 0x1008</code>	Write <code>s0</code>
Write	data0	value	
Write	command	<code>write, postexec, regno = 0x1009</code>	Write <code>s1</code> , then execute program buffer

Write block of memory using program buffer:

Op	Address	Value	Comment
Write	progbuf0	sw s1, 0(s0)	
Write	progbuf1	addi s0, s0, 4	
Write	progbuf2	ebreak	
Write	data0	address	
Write	command	write, regno = 0x1008	Write s0
Write	data0	value0	
Write	command	write, postexec, regno = 0x1009	Write s1, then execute program buffer
Write	abstractauto	autoexecdata [0]	Set autoexecdata [0]
Write	data0	value1	
...	...	...	...
Write	data0	valueN	
Write	abstractauto	0	Clear autoexecdata [0]

B.8.3 Using Abstract Memory Access

Abstract memory accesses act as if they are performed by the hart, although the actual implementation may differ.

Write a word to memory using abstract memory access:

Op	Address	Value	Comment
Write	data1	address	
Write	data0	value	
Write	command	cmdtype=2, aamsize =2, write=1	

Write a block of memory using abstract memory access:

Op	Address	Value	Comment
Write	data1	address	
Write	data0	value0	
Write	command	cmdtype=2, aamsize =2, write=1, aampostincrement =1	
Write	abstractauto	1	Re-execute the command when data0 is accessed
Write	data0	value1	
Write	data0	value2	
...	...	...	...
Write	data0	valueN	
Write	abstractauto	0	Disable auto-exec

B.9 Triggers

A debugger can use hardware triggers to halt a hart when a certain event occurs. Below are some examples, but as there is no requirement on the number of features of the triggers implemented by a hart, these examples may not be applicable to all implementations. When a debugger wants to set a trigger, it writes the desired configuration, and then reads back to see if that configuration is supported.

Enter Debug Mode just before the instruction at 0x80001234 is executed, to be used as an instruction breakpoint in ROM:

tdata1	0x105c	action=1, match=0, m=1, s=1, u=1, execute=1
tdata2	0x80001234	address

Enter Debug Mode right after the value at 0x80007f80 is read:

tdata1	0x4159	timing=1, action=1, match=0, m=1, s=1, u=1, load=1
tdata2	0x80007f80	address

Enter Debug Mode right before a write to an address between 0x80007c80 and 0x80007cef (inclusive):

tdata1 0	0x195a	action=1, chain=1, match=2, m=1, s=1, u=1, store=1
tdata2 0	0x80007c80	start address (inclusive)
tdata1 1	0x11da	action=1, match=3, m=1, s=1, u=1, store=1
tdata2 1	0x80007cf0	end address (exclusive)

Enter Debug Mode right before a write to an address between 0x81230000 and 0x8123fff (inclusive):

tdata1	0x10da	action=1, match=1, m=1, s=1, u=1, store=1
tdata2	0x8123fff	16 bits to match exactly, then 0, then all ones.

Enter Debug Mode right after a read from an address between 0x86753090 and 0x8675309f or between 0x96753090 and 0x9675309f (inclusive):

tdata1 0	0x41a59	timing=1, action=1, chain=1, match=4, m=1, s=1, u=1, load=1
tdata2 0	0xfff03090	Mask for low half, then match for low half
tdata1 1	0x412d9	timing=1, action=1, match=5, m=1, s=1, u=1, load=1
tdata2 1	0xefff8675	Mask for high half, then match for high half

B.10 Handling Exceptions

Generally the debugger can avoid exceptions by being careful with the programs it writes. Sometimes they are unavoidable though, e.g. if the user asks to access memory or a CSR that is not implemented. A typical debugger will not know enough about the platform to know what's going to happen, and must attempt the access to determine the outcome.

When an exception occurs while executing the Program Buffer, `cmderr` becomes set. The debugger can check this field to see whether a program encountered an exception. If there was an exception, it's left to the debugger to know what must have caused it.

B.11 Quick Access

There are a variety of instructions to transfer data between GPRs and the `data` registers. They are either loads/stores or CSR reads/writes. The specific addresses also vary. This is all specified in `hartinfo`. The examples here use the pseudo-op `transfer dest, src` to represent all these options.

Halt the hart for a minimum amount of time to perform a single memory write:

Op	Address	Value	Comment
Write	<code>progbuf0</code>	<code>transfer arg2, s0</code>	Save <code>s0</code>
Write	<code>progbuf1</code>	<code>transfer s0, arg0</code>	Read first argument (address)
Write	<code>progbuf2</code>	<code>transfer arg0, s1</code>	Save <code>s1</code>
Write	<code>progbuf3</code>	<code>transfer s1, arg1</code>	Read second argument (data)
Write	<code>progbuf4</code>	<code>sw s1, 0(s0)</code>	
Write	<code>progbuf5</code>	<code>transfer s1, arg0</code>	Restore <code>s1</code>
Write	<code>progbuf6</code>	<code>transfer s0, arg2</code>	Restore <code>s0</code>
Write	<code>progbuf7</code>	<code>ebreak</code>	
Write	<code>data0</code>	address	
Write	<code>data1</code>	data	
Write	<code>command</code>	<code>0x10000000</code>	Perform quick access

This shows an example of setting the `m` bit in `mcontrol` to enable a hardware breakpoint in M-mode. Similar quick access instructions could have been used previously to configure the trigger that is being enabled here:

Op	Address	Value	Comment
Write	<code>progbuf0</code>	<code>transfer arg0, s0</code>	Save <code>s0</code>
Write	<code>progbuf1</code>	<code>li s0, (1 << 6)</code>	Form the mask for <code>m</code> bit
Write	<code>progbuf2</code>	<code>csrrs x0, tdata1, s0</code>	Apply the mask to <code>mcontrol</code>
Write	<code>progbuf3</code>	<code>transfer s0, arg2</code>	Restore <code>s0</code>
Write	<code>progbuf4</code>	<code>ebreak</code>	
Write	<code>command</code>	<code>0x10000000</code>	Perform quick access

Appendix C

Bug Fixes

C.1 0.13.1

Since the ratification of 0.13, the following bugs have been fixed in 0.13.1:

C.1.1 Resume ack bit is set after resuming

The third paragraph of Section 3.5 has a mistake. At the end of the process described there, the resume ack bit is *set*.

C.1.2 `aamsize` does not affect Argument Width

The Argument Width of the Access Memory abstract command defined in Section 3.7.1.3 is determined by DXLEN, and not by `aamsize`.

C.1.3 `sbddata0` Reads Order of Operations

The order of operations listed in Section 3.12.23, describing reads from `sbddata0`, is incorrect. It should read:

Reads from this register start the following:

1. “Return” the data.
2. Set `sbbusy`.
3. If `sbreadondata` is set, perform another system bus read.
4. If `sbautoincrement` is set, increment `sbaddress`.
5. Clear `sbbusy`.

C.1.4 Hart reset behavior when `haltreq` is set

When a hart comes out of reset and `haltreq` is set, the hart will immediately enter Debug Mode.

C.1.5 `mte` only applies when `action=0`

The definition of `mte` in Section 5.2.6 should state that `mte` only affects triggers whose action is 0.

C.1.6 `sselect` applies to `svalue`

In Section 5.2.13, when `sselect` is 0 it ignores `svalue`.

C.1.7 Last trigger example

In the last example in Section B.9, the value for `tdata2` 1 should be 0xefff8675.

C.2 0.13.2

Fixed a formatting issue that caused step 1 in the Quick Access description to be missing from the document.

Index

aampostincrement, [15](#)
aamsize, [15](#)
aamvirtual, [15](#)
aarpostincrement, [13](#)
aarsize, [13](#)
abits, [65](#)
abstractauto, [29](#)
abstractcs, [27](#)
Access Memory, [14](#)
Access Register, [12](#)
ackhavereset, [23](#)
action, [56](#), [59](#), [60](#)
address, [35](#), [36](#), [65](#)
allhalted, [21](#)
allhavereset, [21](#)
allnonexistent, [21](#)
allresumeack, [21](#)
allrunning, [21](#)
allunavail, [21](#)
anyhalted, [21](#)
anyhavereset, [21](#)
anynonexistent, [21](#)
anyresumeack, [21](#)
anyrunning, [21](#)
anyunavail, [21](#)
authbusy, [22](#)
authdata, [31](#)
authenticated, [22](#)
autoexecdata, [29](#)
autoexecprogbuf, [29](#)

busy, [27](#)
BYPASS, [66](#)

cause, [43](#)
chain, [57](#)
clrresethaltreq, [24](#)
cmderr, [28](#)
cmdtype, [13–15](#), [29](#)
command, [28](#)

confstrptr0, [29](#)
confstrptrvalid, [22](#)
control, [29](#)
count, [58](#)

data, [37](#), [38](#), [50](#), [53](#), [65](#)
data0, [30](#)
dataaccess, [26](#)
dataaddr, [26](#)
datacount, [28](#)
datasize, [26](#)
dcsr, [42](#)
dmactive, [25](#)
dmcontrol, [22](#)
dmi, [65](#)
dmihardreset, [64](#)
dmireset, [64](#)
dmistat, [65](#)
dmode, [50](#)
dmstatus, [20](#)
dpc, [44](#)
dscratch0, [45](#)
dscratch1, [45](#)
dtmcs, [64](#)

ebreakm, [42](#)
ebreaks, [42](#)
ebreaku, [42](#)
etrigger, [60](#)
execute, [58](#)

field, [2](#)

haltreq, [23](#)
haltsum0, [31](#)
haltsum1, [31](#)
haltsum2, [32](#)
haltsum3, [32](#)
hartinfo, [25](#)
hartreset, [23](#)
hartsel, [22](#)

- hartselhi, [24](#)
- hartsello, [24](#)
- hasel, [24](#)
- hasresethaltreq, [22](#)
- hawindow, [26](#)
- hawindowssel, [26](#)
- hit, [54](#), [58–60](#)

- icount, [58](#)
- IDCODE, [63](#)
- idle, [64](#)
- impebreak, [21](#)
- info, [51](#)
- itrigger, [59](#)

- load, [58](#)

- m, [57–60](#)
- ManufId, [64](#)
- maskmax, [54](#)
- match, [57](#)
- mcontext, [52](#)
- mcontrol, [53](#)
- mprven, [43](#)
- mpte, [52](#)
- mselect, [61](#)
- mte, [52](#)
- mvalue, [61](#)

- ndmreset, [24](#)
- nextdm, [30](#)
- nmip, [43](#)
- nscratch, [25](#)

- op, [66](#)

- PartNumber, [64](#)
- postexec, [13](#)
- priv, [45](#)
- progbuf0, [30](#)
- progbufsize, [27](#)
- prv, [44](#), [46](#)

- Quick Access, [14](#)

- regno, [13](#)
- resethaltreq, [22](#)
- resumereq, [23](#)

- s, [57–60](#)
- sbaccess, [33](#)
- sbaccess128, [34](#)
- sbaccess16, [34](#)
- sbaccess32, [34](#)
- sbaccess64, [34](#)
- sbaccess8, [34](#)
- sbaddress0, [34](#)
- sbaddress1, [35](#)
- sbaddress2, [35](#)
- sbaddress3, [36](#)
- sbasize, [34](#)
- sbautoincrement, [33](#)
- sbbusy, [33](#)
- sbbusycerror, [33](#)
- sbc, [32](#)
- sbddata0, [36](#)
- sbddata1, [37](#)
- sbddata2, [37](#)
- sbddata3, [38](#)
- sberror, [34](#)
- sbreadonaddr, [33](#)
- sbreadondata, [33](#)
- sbversion, [33](#)
- scontext, [52](#)
- select, [54](#)
- setresethaltreq, [24](#)
- shortname, [2](#)
- sizehi, [54](#)
- sizelo, [56](#)
- sselect, [61](#)
- step, [44](#)
- stepie, [42](#)
- stopcount, [43](#)
- stoptime, [43](#)
- store, [58](#)
- svalue, [61](#)

- target-specific, [15](#)
- tcontrol, [51](#)
- tdata1, [50](#)
- tdata2, [50](#)
- tdata3, [51](#)
- textra32, [60](#)
- textra64, [61](#)
- timing, [55](#)
- tinfo, [51](#)
- transfer, [13](#)
- tselect, [49](#)

type, [50](#)

u, [57–60](#)

Version, [63](#)

version, [22](#), [65](#)

write, [13](#), [15](#)

xdebugver, [42](#)